# Laboratorio 9
# Modelos generativos

**Curso:** Deep Learning 10

**Autores:** Diego Lopez, Ricardo Godinez, Vianka Castro y Felipe Aguilar

## Informe final — Tasks 1–4

1. VAE: reconstrucción, espacio latente y análisis cuantitativo.
2. Difusión forward: verificación, calendarios y relación señal a ruido.
3. Difusión condicional: U-Net, muestreo y guía sin clasificador.
4. Comparación VAE–difusión y estimación de pass@k.

Semilla de los experimentos: **3045**. Este documento reúne las respuestas, tablas y figuras guardadas en los cuatro notebooks. El código comentado se entrega en esos notebooks y en el repositorio; no se repitieron los entrenamientos para compilar este informe.

La Task 4 documenta una nueva ejecución del VAE β=1 y mide ambos tiempos en CPU. Sus valores no sustituyen los de la Task 1 ni los tiempos de GPU de la Task 3.

> **Alcance de la evidencia:** la Task 2.5(c) existente calcula la varianza con 10 000 imágenes de entrenamiento, no con todo el conjunto solicitado. Se conserva ese resultado y se identifica su alcance, sin presentarlo como una medición completa. La interpretación inicial del SNR alto en la Task 2.5(b) se contrasta y corrige con los resultados de la Task 3.5(d).

# Laboratorio 9 - Task 1: beta-VAE sobre Fashion-MNIST

La idea central es entrenar la misma arquitectura VAE con $\beta \in \{0.1, 1, 10\}$ y medir cómo cambia el compromiso entre reconstrucción e información latente.

## Explicación Inicial Conceptos

En esta primera parte del laboratorio se implementa y analiza un Autoencoder Variacional (VAE)

- es un modelo de aprendizaje profundo que pertenece a la familia de los modelos generativos. Su objetivo es aprender una representación latente de los datos de entrada, que permita generar nuevas muestras similares a las observadas. En particular, se estudia la variante $\beta$-VAE, que introduce un parámetro $\beta$ para controlar el balance entre la calidad de reconstrucción y la regularización del espacio latente.

Estamos utilizando el conjunto Fashion-MNIST con el objetivo de estudiar cómo cambia el comportamiento del modelo al modificar el parámetro $\beta$, que controla el balance entre la calidad de *reconstrucción* y la *regularización* del espacio latente.

Para ello, se entrenan tres modelos con:

- $\beta = 0.1$
- $\beta = 1$
- $\beta = 10$

A partir de estos modelos se evalúan las curvas de reconstrucción y divergencia KL, la organización del espacio latente, la capacidad de generación de nuevas imágenes y las interpolaciones entre ejemplos. Por último, se comparan cuantitativamente los resultados mediante métricas como error de reconstrucción, KL en bits y nitidez, con el objetivo de entender el efecto que tiene $\beta$ sobre la información contenida en la representación latente y sobre la calidad visual de las imágenes generadas.

## 1. Task 1.1 - Configuración Experimental

`ToTensor()` convierte los píxeles de 0-255 a `float` en $[0, 1]$. No usamos `random_split`: el enunciado pide explícitamente tomar **los últimos 5,000** ejemplos del conjunto de entrenamiento como validación.

**Configuración:** batch_size=128, latent_dim=2, epochs=20, lr=0.001, β∈{0.1, 1, 10}. Arquitectura compartida: 784→400→(μ, log σ²), con dos coordenadas latentes; decoder 2→400→784. Se minimiza Lβ = SSE + β·KL, con SSE sumada sobre los 784 píxeles.

```
train / val / test: 55000 5000 10000
forma del lote: (128, 1, 28, 28) (128,)
rango de píxeles: 0.0 1.0
```

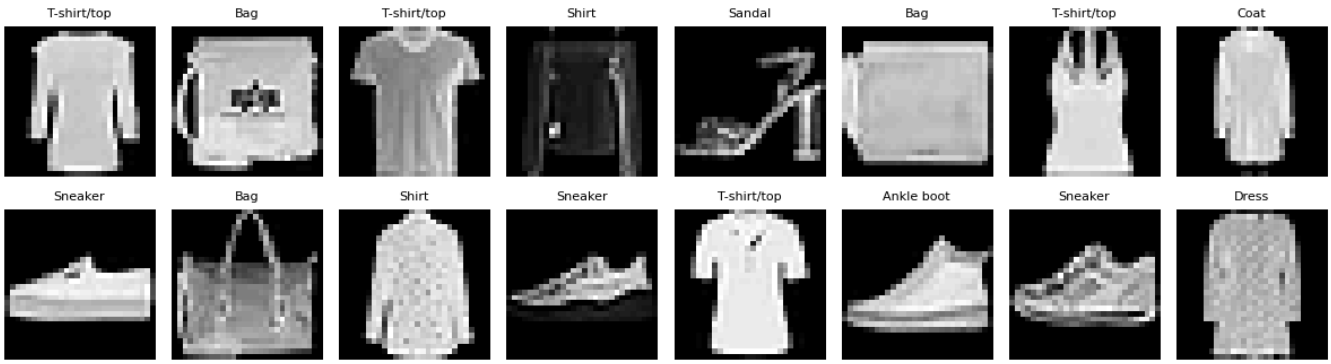



Figura 1. Task 1 · salida de la celda 6 de 01_vae_fashion_mnist.ipynb.

## Task 1.2 - Implementación del VAE y entrenamiento

El encoder produce $\mu$ y $\log \sigma^2$. La reparametrización se escribe directamente:

$$z = \mu + \exp(\tfrac{1}{2}\log \sigma^2) \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

La KL cerrada por imagen, sumada en las dos dimensiones latentes, es

$$D_{KL}(q\|p) = -\frac{1}{2}\sum_j \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\right).$$

La reconstrucción se suma sobre los 784 píxeles y luego se promedia sobre el lote. Esta convención evita cambiar accidentalmente la escala relativa de reconstrucción y KL.

### Entrenamiento

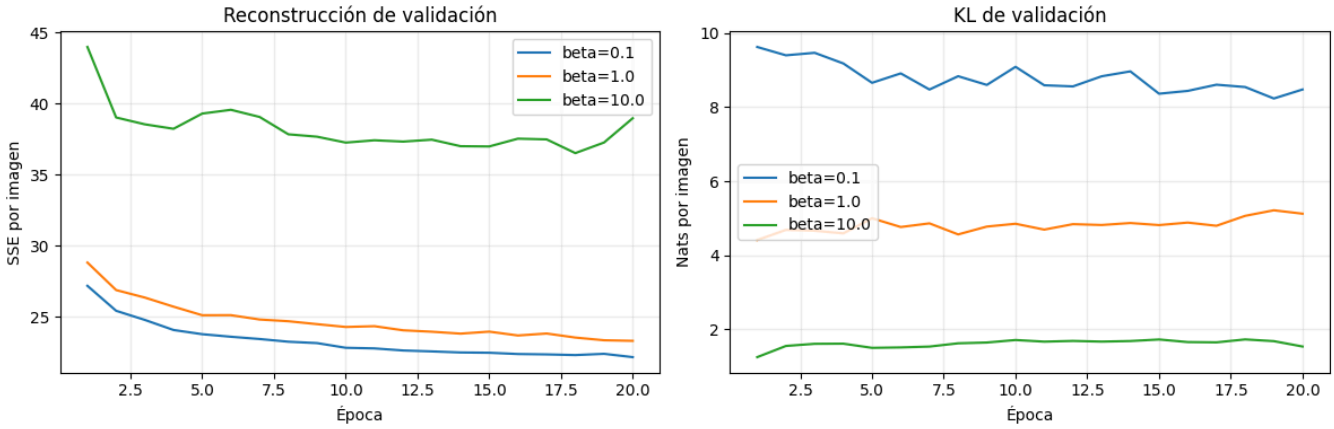



Figura 2. Task 1 · salida de la celda 12 de 01_vae_fashion_mnist.ipynb.

## Task 1.3 - Análisis Visual del espacio latente y generación de muestras

Las celdas siguientes usan únicamente resultados de los tres modelos entrenados. Cada figura se guarda también en `outputs/task1/`. En este análisis podremos observar cómo el parámetro $\beta$ afecta la organización del espacio latente y la calidad de las imágenes generadas. Además de las visualizaciones, se calculan métricas

cuantitativas como el error de reconstrucción, la divergencia KL y la nitidez de las imágenes generadas, para comparar los modelos entrenados con diferentes valores de $\beta$.

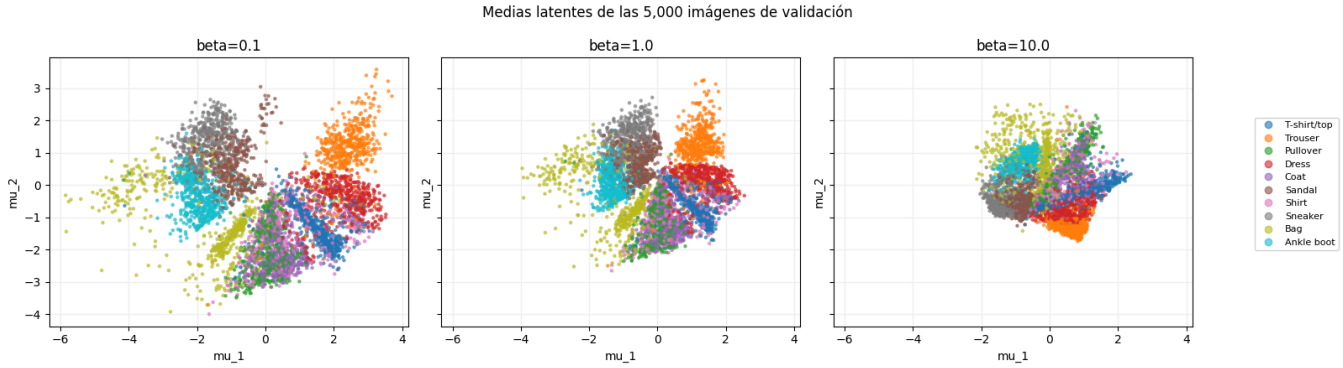



Figura 3. Task 1 · salida de la celda 14 de 01_vae_fashion_mnist.ipynb.

En el diagrama de dispersión se observa el efecto de $\beta$ sobre la organización del espacio latente:

- $\beta = 0.1$: las medias $\mu$ ocupan una región amplia ($\mu_1$ llega hasta cerca de $-6$ y $\mu_2$ hasta cerca de $-4$), mucho más grande que la zona donde $\mathcal{N}(0, I)$ concentra su probabilidad. Las clases forman grupos separados por zonas vacías.
- $\beta = 1$: la nube se contrae aproximadamente a $[-4,\ 2.5] \times [-2.5,\ 3.3]$. Los grupos siguen siendo distinguibles (Trouser, Bag, el calzado y las prendas superiores ocupan regiones distintas), aunque Pullover, Coat y Shirt se traslapan entre sí.
- $\beta = 10$: la nube se comprime aproximadamente a $[-2,\ 2.4] \times [-2,\ 2.5]$ y casi todas las clases se superponen. Solo Trouser y Bag conservan una zona algo separada.

Esto es consistente con que $\beta$ multiplica el término KL, que penaliza alejar $\mu$ del origen: a mayor $\beta$, más se comprimen las medias hacia el centro y menos espacio queda para separar las clases.

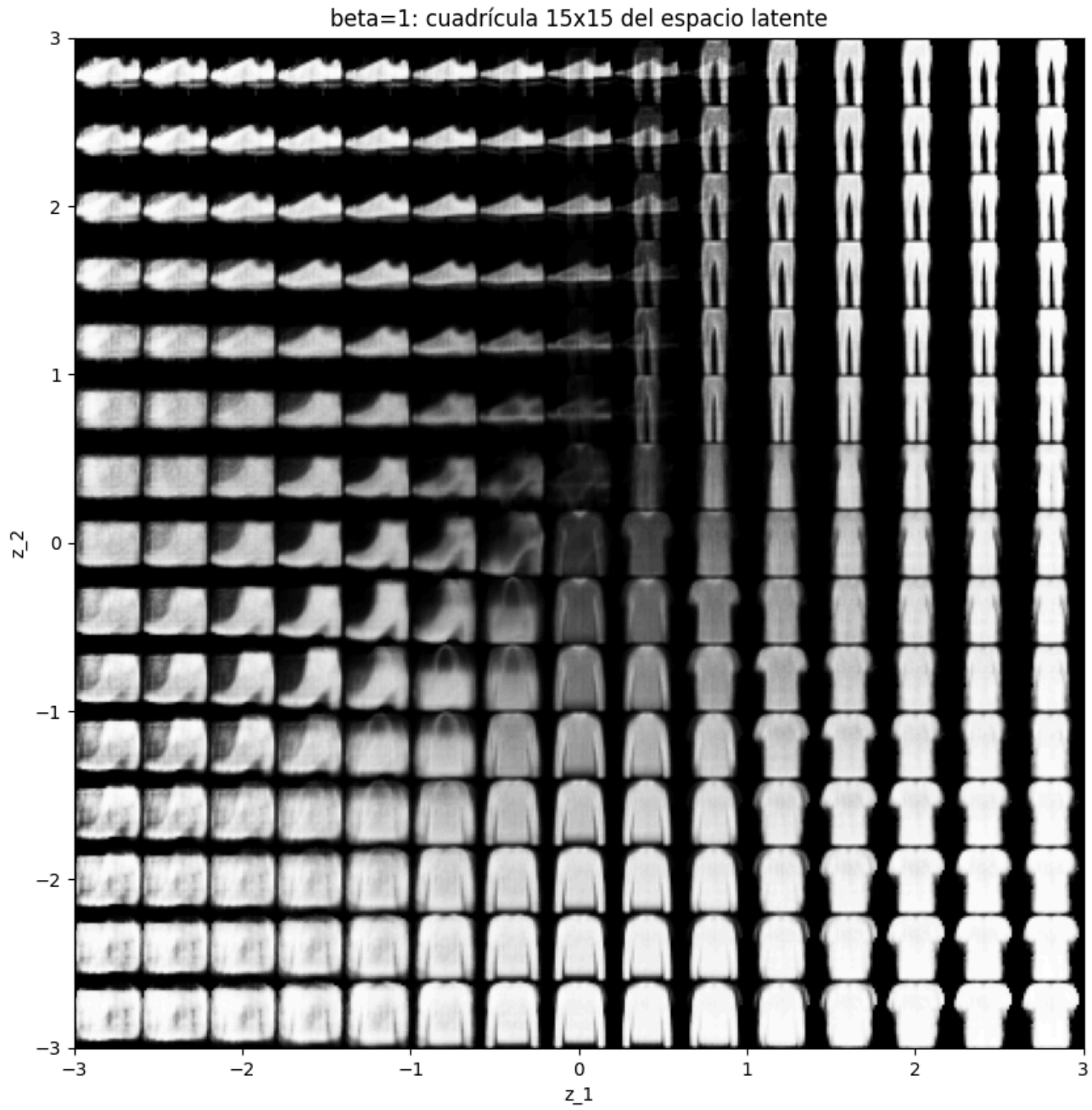



Figura 4. Task 1 · salida de la celda 16 de 01_vae_fashion_mnist.ipynb.

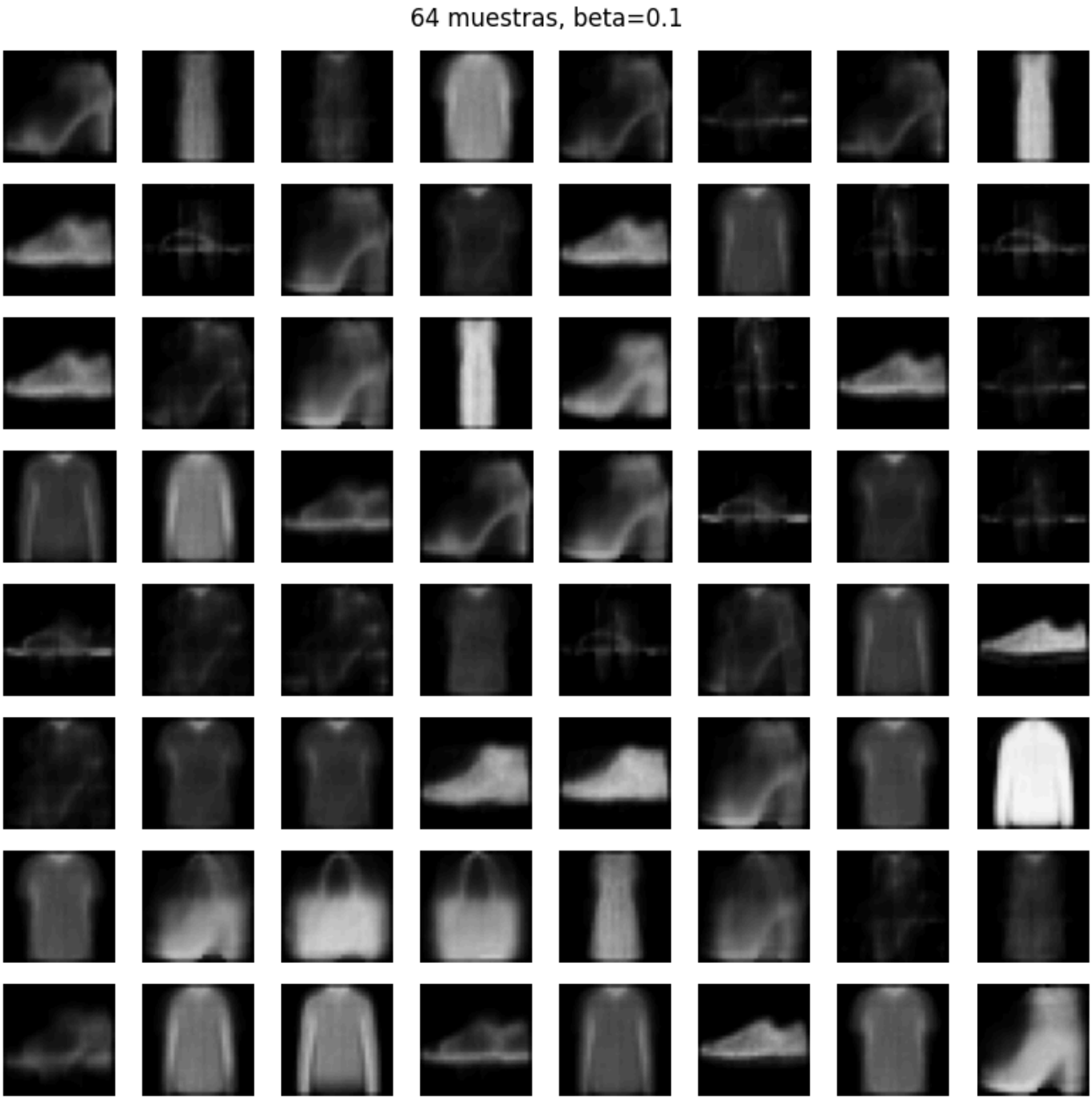



Figura 5. Task 1 · salida de la celda 17 de 01_vae_fashion_mnist.ipynb.

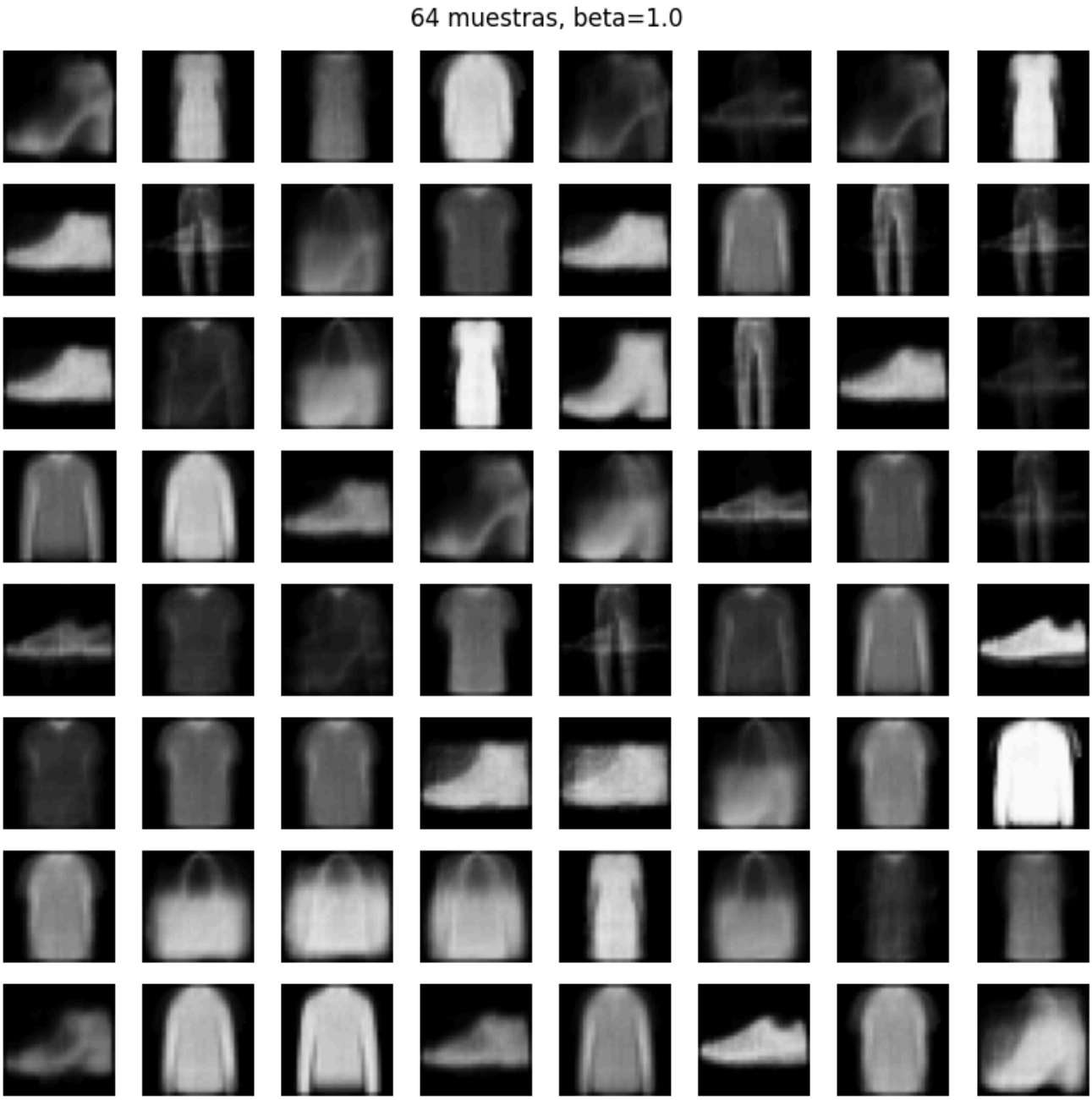



Figura 6. Task 1 · salida de la celda 17 de 01_vae_fashion_mnist.ipynb.

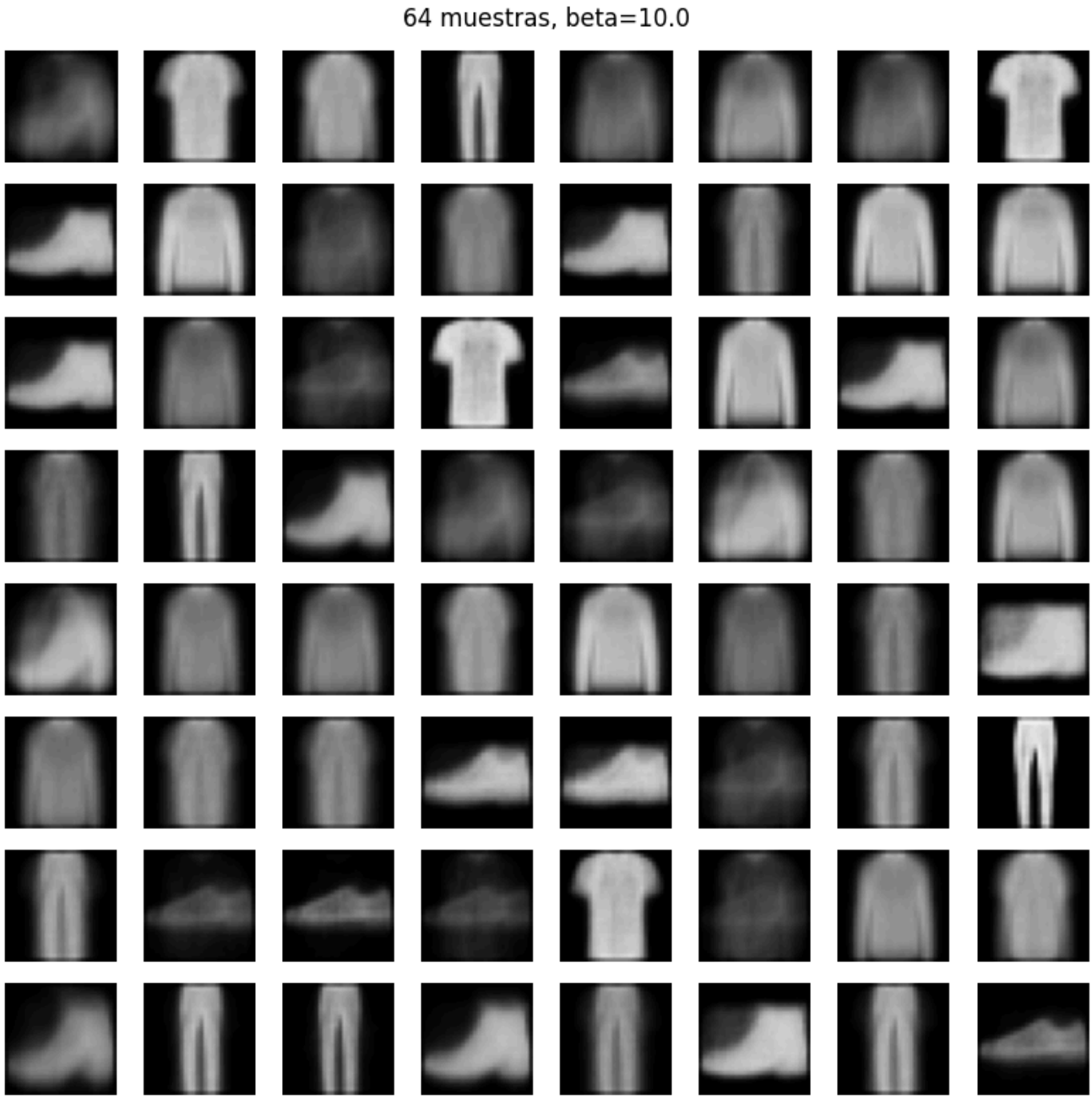



Figura 7. Task 1 · salida de la celda 17 de 01_vae_fashion_mnist.ipynb.

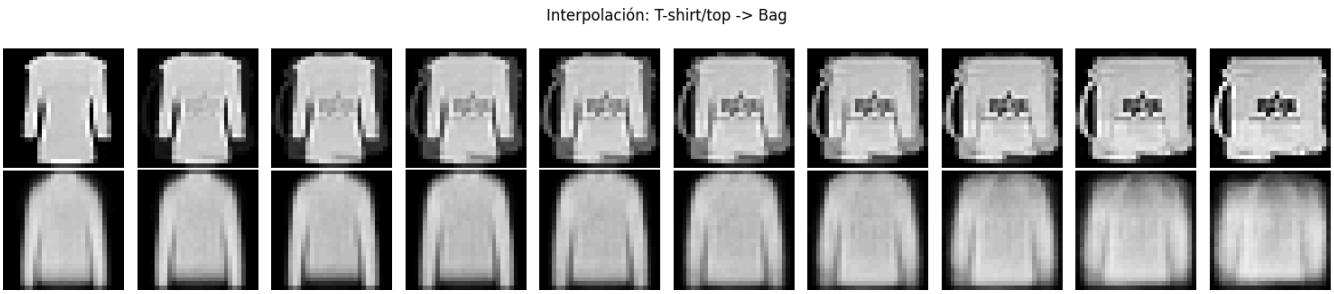



Figura 8. Task 1 · salida de la celda 18 de 01_vae_fashion_mnist.ipynb.

En la interpolación entre una camiseta y una bolsa, la interpolación directa en píxeles genera imágenes intermedias que parecen una superposición de ambos objetos. En el punto medio todavía se observan simultáneamente características de la camiseta y de la bolsa. En cambio, la interpolación en el espacio latente produce una transición más suave y estructurada, ya que el decoder transforma puntos intermedios del espacio aprendido en imágenes que permanecen dentro de la distribución aprendida por el modelo.

La diferencia se debe a que la interpolación en píxeles combina intensidades independientemente de la estructura semántica de la imagen, mientras que la interpolación latente se realiza sobre representaciones comprimidas que contienen características aprendidas de las prendas.

## 4. Task 1.4 - Evaluación Cuantitativa e Interpretación

Estas celdas producen la evidencia numérica que respalda las respuestas de la Task 1.4.

Para un decoder gaussiano con varianza fija $\sigma_x^2$, la parte dependiente de la reconstrucción en la ELBO negativa es

$$\frac{1}{2\sigma_x^2}\|x-\hat{x}\|^2 + D_{KL}.$$

Multiplicar toda la expresión por $2\sigma_x^2$ no cambia su óptimo y da

$$\|x-\hat{x}\|^2 + 2\sigma_x^2 D_{KL}.$$

Al compararla con $\mathcal{L}_\beta = \|x-\hat{x}\|^2 + \beta D_{KL}$ se obtiene $\beta = 2\sigma_x^2$ y, por tanto, $\sigma_x^2 = \beta/2$.

```
beta |  Recon final |  KL (nats) |  KL (bits)
-----------------------------------------------
   0.1 |      22.1864 |     8.4781 |    12.2314
   1.0 |      23.3299 |     5.1227 |     7.3905
  10.0 |      38.9694 |     1.5345 |     2.2139
Referencia log2(10): 3.3219 bits
```

Al aumentar β, el término KL recibe un mayor peso y obliga a la distribución latente a acercarse más a la normal estándar. Esto se observa claramente en los resultados: la KL disminuye de 12.2314 bits para β=0.1 a 2.2139 bits para β=10. Sin embargo, esta mayor regularización tiene un costo en reconstrucción, ya que el error aumenta de 22.1864 a 38.9694. Para β=10, la KL se encuentra por debajo de los 3.3219 bits requeridos teóricamente para identificar una de diez clases, por lo que el código latente no dispone, en promedio, de suficiente capacidad informativa para codificar perfectamente la clase.

Este resultado coincide con el diagrama de dispersión de β=10, donde las clases ocupan una región mucho más compacta y presentan una superposición considerable. Aunque todavía existen zonas asociadas con determinadas clases, la separación es notablemente menor que para β=0.1 y β=1.0.

```
closed_form: 5.08353710
monte_carlo: 5.08533287
relative_error: 0.00035325
standard_error: 0.00316382
error_in_standard_errors: 0.56759548
```

Para verificar la expresión cerrada de la divergencia KL se tomó una imagen de validación con el modelo β=1 y se comparó el valor analítico con una estimación Monte Carlo utilizando 10^5 muestras. La KL cerrada fue 5.08353710, mientras que la estimación Monte Carlo fue 5.08533287. El error relativo fue únicamente 0.00035325, equivalente aproximadamente a 0.57 errores estándar. Por lo tanto, la diferencia observada es completamente compatible con la variabilidad estadística esperada de una estimación Monte Carlo de este tamaño y confirma numéricamente la implementación de la fórmula cerrada.

### Task 1.4c - Regiones no reconocibles e interpolaciones

**Región no reconocible.** En la cuadrícula de $\beta = 1$ hay dos zonas donde el decoder no produce prendas reconocibles:

1. La franja vertical $z_1 \in [0,\ 0.5]$, $z_2 \in [0.5,\ 3]$: las imágenes son siluetas tenues y semitransparentes que mezclan un tenis con un pantalón.
2. La esquina inferior izquierda, $z_1 \leq -2$ y $z_2 \leq -1.5$: aparecen manchas rectangulares con textura ruidosa, sin la forma clara de un bolso ni de un botín.

Al compararlas con el diagrama de dispersión de $\beta = 1$, ambas zonas casi no contienen medias $\mu$ de imágenes reales. La primera es la frontera vacía entre el grupo de Sneaker ($\mu \approx (-1,\ 2)$) y el de Trouser ($\mu \approx (1,\ 1.2)$); la segunda queda fuera de la nube de puntos, debajo de la cola de Bag. Durante el entrenamiento el decoder solo recibe valores de $z$ cercanos a las $\mu$ de los datos (más el ruido $\sigma \odot \epsilon$), así que nunca aprendió qué dibujar en esas zonas y devuelve mezclas.

La explicación está en el término KL, que penaliza dos cosas: alejar $\mu$ del origen y usar $\sigma$ pequeñas. Con $\beta = 1$ esa penalización es moderada (KL final de 7.39 bits, más del doble de los 3.32 bits de la clase), por lo que al encoder le conviene separar las clases en grupos con huecos entre ellos para reconstruir mejor, en lugar de cubrir todo el prior. Pero $\mathcal{N}(0, I)$ sí asigna probabilidad a esos huecos (la franja $z_1 \in [0,\ 0.5]$ está a menos de una desviación estándar del origen), y por eso varias de las 64 muestras de la Task 1.3c también salen como siluetas tenues. Con $\beta = 10$ los huecos se cierran porque la KL obliga a que las distribuciones $q(z \mid x)$ se traslapen (ver dispersión), y con $\beta = 0.1$ los huecos son todavía más grandes.

**Interpolaciones.** En el punto medio de la interpolación en píxeles se ve una superposición fantasma: la camiseta sigue visible y encima aparecen, a media intensidad, el asa y el estampado del bolso. Es un promedio de intensidades, no una prenda real. En el punto medio de la interpolación latente aparece una sola prenda coherente y borrosa, con forma de suéter de manga larga y sin asa.

Difieren porque la interpolación latente pasa por el decoder, que solo produce imágenes parecidas a las de entrenamiento, mientras que la interpolación en píxeles mezcla intensidades sin importar si el resultado pertenece a la distribución de los datos. Además, en el diagrama de dispersión de $\beta = 1$ las clases Pullover, Coat y Shirt se ubican entre los grupos de T-shirt/top y Bag, lo que explica que el punto medio latente parezca un suéter y no una mezcla de camiseta y bolso. También se nota que los extremos de la fila latente no reproducen exactamente las imágenes originales (la camiseta aparece con mangas largas y el bolso pierde su estampado), algo esperable con un código de solo 2 dimensiones que transmite 7.39 bits.

```
conjunto |    nitidez |   sigma_x^2=beta/2
-----------------------------------------------------
          reales |   0.089146 |                  -
    VAE beta=0.1 |   0.031240 |              0.050
    VAE beta=1.0 |   0.046194 |              0.500
   VAE beta=10.0 |   0.044091 |              5.000
```

### Task 1.4d - Nitidez y relación con $\sigma_x^2$

La nitidez promedio de las 1 000 imágenes reales fue **0.089146**, mientras que las 1 000 generadas por cada VAE obtuvieron **0.031240** ($\beta = 0.1$), **0.046194** ($\beta = 1$) y **0.044091** ($\beta = 10$). Todas las muestras generadas tienen entre 35% y 52% de la nitidez de las imágenes reales, lo que confirma cuantitativamente que son borrosas.

**Relación con $\sigma_x^2$.** Según la derivación de la sección anterior, nuestra pérdida corresponde a un decoder gaussiano con $\sigma_x^2 = \beta/2$. Para $\beta = 1$ eso significa suponer un ruido de observación con varianza 0.5, es decir, una desviación estándar de $\sqrt{0.5} \approx 0.71$, en píxeles que solo toman valores en $[0, 1]$. Para el modelo, casi cualquier diferencia fina entre píxeles vecinos (bordes, costuras, estampados) es indistinguible de ese ruido, así que la mejor predicción ante la incertidumbre es la media condicional $\mathbb{E}[x \mid z]$: el promedio de todas las prendas compatibles con ese código. Promediar prendas ligeramente distintas cancela los bordes finos y reduce la diferencia entre píxeles adyacentes, que es justo lo que mide nuestra nitidez. Además, con solo 2 dimensiones latentes y una KL de 7.39 bits, un mismo $z$ corresponde a muchas imágenes distintas, lo que hace ese promedio todavía más borroso. Incluso con $\beta = 0.1$ ($\sigma_x^2 = 0.05$, desviación $\approx 0.22$) el ruido supuesto sigue siendo grande comparado con el contraste de los detalles finos, por lo que ningún valor de $\beta$ elimina el problema.

**¿Por qué $\beta = 0.1$ es el menos nítido?** Si la nitidez dependiera solo de $\sigma_x^2$, $\beta = 0.1$ debería ser el más nítido, pero obtuvo el valor más bajo (0.031240). La razón está en el espacio latente: con $\beta = 0.1$ las medias $\mu$ ocupan una región mucho más grande que $\mathcal{N}(0, I)$ y los grupos de clases están separados por huecos. Al muestrear $z \sim \mathcal{N}(0, I)$, muchas muestras caen en esos huecos y el decoder genera siluetas oscuras y tenues; en la figura de 64 muestras de $\beta = 0.1$ abundan figuras fantasma tipo sandalia o pantalón con muy poco contraste. Una imagen casi negra tiene diferencias horizontales muy pequeñas, lo que baja el promedio de nitidez.

Con $\beta = 1$ el latente se ajusta mejor al prior y menos muestras caen en huecos, por eso obtiene la mayor nitidez entre los modelos (0.046194). Con $\beta = 10$ casi no hay huecos, pero $\sigma_x^2 = 5$ hace que el decoder promedie todavía más y las prendas pierden detalle (0.044091). La nitidez de las muestras resulta así de dos efectos opuestos: un $\beta$ pequeño da un decoder más detallado pero un latente con huecos, y un $\beta$ grande da un latente sin huecos pero un decoder que promedia más.

# Conclusiones Task 1

Podemos notar que los experimentos muestran claramente el compromiso controlado por $\beta$ en el VAE. Con $\beta = 0.1$ se obtuvo el menor error de reconstrucción (22.1864), pero una KL elevada de 12.2314 bits, esto nos dice que el espacio latente conserva una mayor cantidad de información sobre las entradas.

Ya cuando aumentamos $\beta$, la KL disminuyó y el espacio latente se volvió más compacto y regularizado, pero el error de reconstrucción aumentó. El caso más extremo fue $\beta = 10$, con una KL de 2.2139 bits y un error de reconstrucción de 38.9694.

Para $\beta = 10$, la KL se encuentra por debajo de $\log_2(10) \approx 3.3219$ bits. Esto sugiere que el código latente no contiene, en promedio, suficiente información para identificar perfectamente una de las diez clases. Esta conclusión es consistente con el diagrama de dispersión, donde se observa una mayor superposición entre las clases para $\beta = 10$.

La verificación Monte Carlo confirmó la implementación de la divergencia KL. El valor cerrado fue 5.08353710 y la estimación con $10^5$ muestras fue 5.08533287, con un error relativo de 0.00035325. La diferencia corresponde únicamente a aproximadamente 0.57 errores estándar, por lo que es compatible con la variabilidad esperada del método Monte Carlo.

La cuadrícula del espacio latente de $\beta = 1$ mostró regiones asociadas a distintos tipos de prendas, pero también zonas no reconocibles (la franja $z_1 \in [0,\ 0.5]$, $z_2 \in [0.5,\ 3]$ y la esquina inferior izquierda) que coinciden con huecos del diagrama de dispersión: con $\beta = 1$ la KL no es lo bastante fuerte para que las distribuciones de cada imagen cubran todo el prior. En la interpolación de camiseta a bolso, el punto medio en píxeles es una superposición fantasma de ambas imágenes, mientras que el punto medio latente es una prenda coherente con forma de suéter, porque el decoder solo produce imágenes parecidas a las de entrenamiento.

Por último, todas las muestras del VAE son menos nítidas que las imágenes reales (entre 0.031 y 0.046, frente a 0.089). La pérdida usada equivale a un decoder gaussiano con $\sigma_x^2 = \beta/2$, que supone un ruido grande y lleva al decoder a devolver el promedio de las prendas posibles. Que $\beta = 0.1$ sea el menos nítido muestra que la calidad de las muestras también depende de qué tan bien cubre el espacio latente al prior $\mathcal{N}(0, I)$.

# Laboratorio 9 - Task 2: Proceso forward de difusión

En esta parte construimos el proceso forward de un modelo de difusión: la receta que toma una imagen limpia $x_0$ y la va ensuciando con ruido gaussiano durante $T = 1000$ pasos hasta que no queda nada de ella. Verificamos numéricamente la forma cerrada del atajo, visualizamos cómo se destruye la imagen, medimos cuánta señal queda con el SNR y comparamos el calendario lineal contra uno de coseno diseñado por nosotros.

## 1. Task 2.1 - Datos en $[-1, 1]$

`ToTensor()` deja los píxeles en $[0, 1]$ y `Normalize(0.5, 0.5)` aplica $x \mapsto (x - 0.5)/0.5 = 2x - 1$, que los lleva a $[-1, 1]$. Este rango no es un capricho: el ruido $\varepsilon \sim \mathcal{N}(0, 1)$ está centrado en 0, así que queremos que la imagen también lo esté. Al final del proceso forward la imagen debe parecerse a $\mathcal{N}(0, I)$, y eso solo encaja si los datos viven en una escala comparable.

```
forma de una imagen: (1, 28, 28)
mínimo: -1.000 | máximo: 1.000
media: -0.432 | varianza: 0.500
fracción de píxeles exactamente en -1 (fondo): 0.507
```

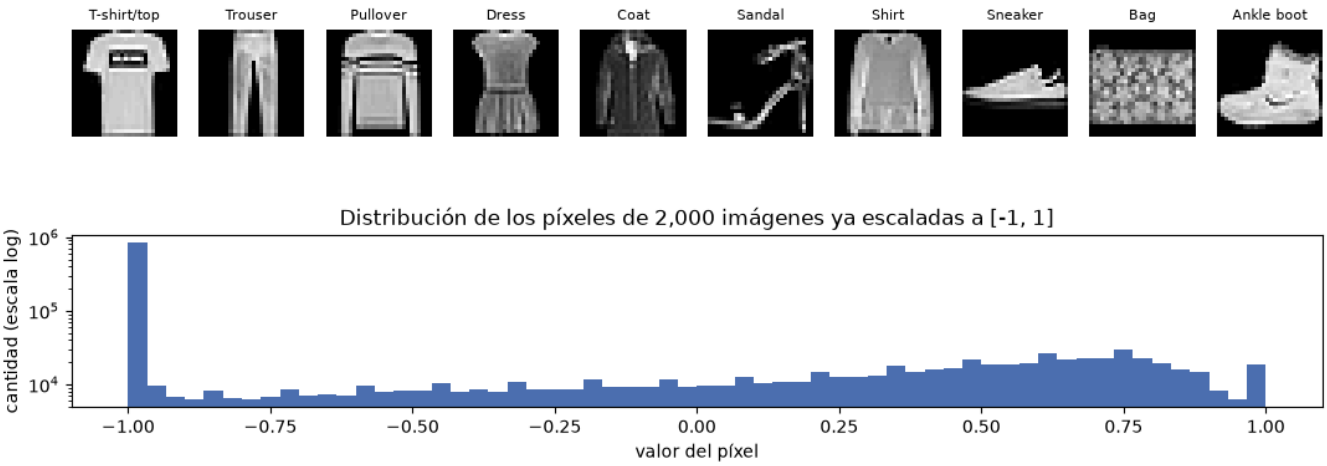



Figura 9. Task 2 · salida de la celda 5 de 02_difusion_forward.ipynb.

Los píxeles quedaron exactamente en $[-1, 1]$. La media es $-0.432$ y no 0 porque cerca de la mitad de cada imagen (50.7 % de los píxeles) es fondo negro, que ahora vale $-1$; eso se ve en el pico gigante de la izquierda del histograma. La varianza de los datos es $0.500$. Este número lo retomamos en la Task 2.5c, porque la varianza de $x_t$ debe pasar de la de los datos ($\approx 0.5$) a la del ruido puro (1) a medida que avanza el proceso.

## 2. Task 2.1 - Calendario lineal de ruido

En cada paso $t$ el proceso forward agrega un poco de ruido según

$$q(x_t \mid x_{t-1}) = \mathcal{N}\big(\sqrt{\alpha_t}\, x_{t-1},\ \beta_t I\big), \qquad \alpha_t = 1 - \beta_t.$$

$\beta_t$ es cuánto ruido nuevo entra en el paso $t$ y $\alpha_t$ es cuánto de la imagen anterior se conserva. El calendario lineal reparte $\beta_t$ en 1000 valores igualmente espaciados entre $\beta_1 = 10^{-4}$ y $\beta_T = 0.02$. La cantidad que realmente importa es el producto acumulado

$$\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s,$$

que mide cuánto de la imagen original $x_0$ sobrevive después de $t$ pasos.

Los tensores de PyTorch empiezan en el índice 0, pero en las fórmulas $t$ va de 1 a $T$. Usamos la convención `tensor[t - 1]` para el paso $t$. Guardamos el calendario en `float64` porque $\bar{\alpha}_t$ llega a valores muy pequeños y más adelante tomamos logaritmos del SNR.

```
formas: (1000,) (1000,) (1000,)
    t |    beta_t |   alpha_t |  alpha_bar_t
-------------------------------------------
    1 |  0.000100 |  0.999900 | 9.999000e-01
    2 |  0.000120 |  0.999880 | 9.997801e-01
   10 |  0.000279 |  0.999721 | 9.981052e-01
  100 |  0.002072 |  0.997928 | 8.970181e-01
  250 |  0.005060 |  0.994940 | 5.240854e-01
  500 |  0.010040 |  0.989960 | 7.858724e-02
  750 |  0.015020 |  0.984980 | 3.350550e-03
 1000 |  0.020000 |  0.980000 | 4.035830e-05
```

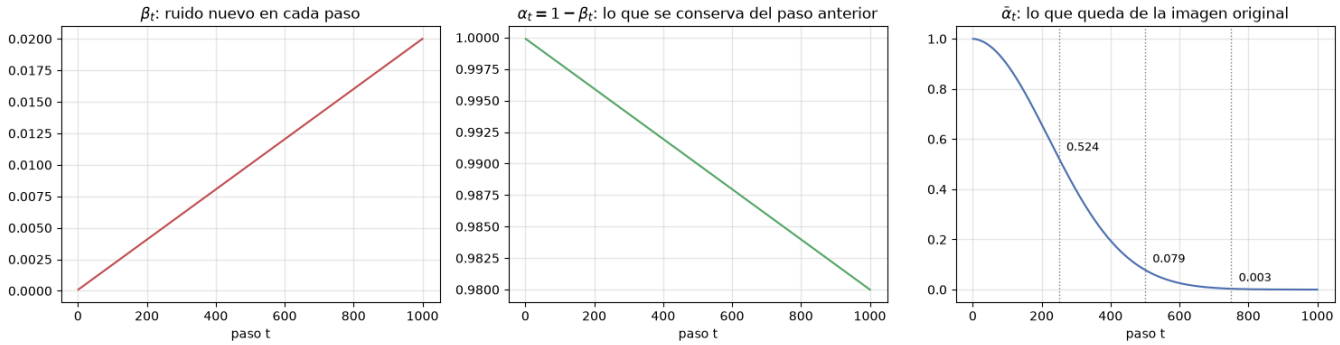



Figura 10. Task 2 · salida de la celda 9 de 02_difusion_forward.ipynb.

Obtuvimos tres tensores de 1000 valores. $\beta_t$ y $\alpha_t$ son rectas, pero $\bar{\alpha}_t$ no lo es: aunque cada $\alpha_t$ está siempre entre 0.98 y 0.9999, multiplicar mil números apenas menores que 1 termina en casi cero. Al final del calendario queda $\bar{\alpha}_T = 4.04 \times 10^{-5}$ de la imagen original.

La curva de $\bar{\alpha}_t$ cae despacio al principio ($\bar{\alpha}_{100} = 0.897$) porque los primeros $\beta_t$ son diminutos, baja rápido en el tramo medio ($0.524$ en $t = 250$ y $0.079$ en $t = 500$) y para $t = 750$ ya solo queda $0.003$. Eso sugiere que el último cuarto de los pasos trabaja sobre algo que ya es casi puro ruido; lo medimos con el SNR en la Task 2.3 y lo atacamos con el calendario de coseno en la 2.4.

## 3. Task 2.1 - El atajo en forma cerrada

Aplicar el ruido paso por paso para llegar a $t = 700$ exigiría 700 operaciones por imagen. La forma cerrada permite saltar directamente de $x_0$ a cualquier $x_t$ con un solo ruido:

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I).$$

El primer término es la **señal** (la imagen original encogida) y el segundo es el **ruido** acumulado. Los coeficientes cumplen $(\sqrt{\bar{\alpha}_t})^2 + (\sqrt{1-\bar{\alpha}_t})^2 = 1$, así que conforme la imagen pierde peso, el ruido gana exactamente lo que ella pierde.

La función `q_sample` acepta un `t` entero o un tensor con un `t` distinto por imagen del lote, que es lo que necesitaremos en la Task 3 para entrenar la U-Net. Devuelve también el ruido $\varepsilon$ usado, porque ese será el objetivo que la red debe predecir.

```
t=1:    |x_1 - x_0| máximo = 0.0332  (casi la imagen limpia)
t=1000: |x_T - eps| máximo = 0.0064  (casi ruido puro)
lote con un t distinto por imagen: (4, 1, 28, 28) con t = [1, 250, 500, 1000]
```

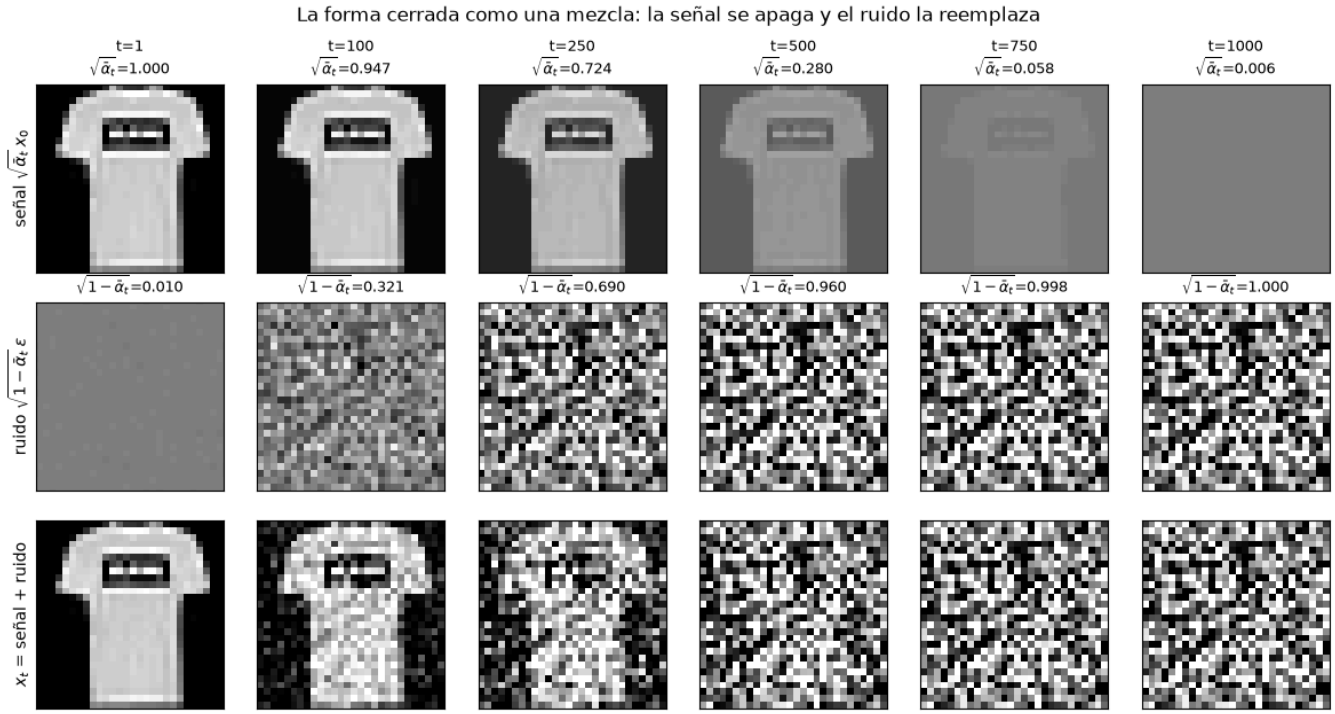



Figura 11. Task 2 · salida de la celda 13 de 02_difusion_forward.ipynb.

Las pruebas confirman los dos extremos: en $t = 1$ el cambio máximo de un píxel es apenas $0.033$, así que $x_1$ es prácticamente la imagen limpia, y en $t = 1000$ la diferencia entre $x_T$ y el ruido es como mucho $0.006$, así que $x_T$ es prácticamente ruido puro. La función también funciona con un lote donde cada imagen tiene su propio $t$.

La figura muestra la fórmula como una mezcla de dos ingredientes. La fila de arriba es la camiseta multiplicada por $\sqrt{\bar{\alpha}_t}$: no se borra, se va apagando hacia el gris (el 0). La fila del medio es el mismo ruido multiplicado por $\sqrt{1-\bar{\alpha}_t}$, que va subiendo de volumen. La fila de abajo es su suma. En $t = 500$ la camiseta todavía se reconoce en la fila de la señal, pero en $x_t$ ya no, porque su peso ($0.280$) es mucho menor que el del ruido ( $0.960$). Las imágenes se muestran recortadas a $[-1, 1]$ para usar la misma escala de grises en todas.

## 4. Task 2.2 - Verificación numérica del atajo

El atajo dice que, después de $t$ pasos, $x_t \mid x_0 \sim \mathcal{N}\left(\sqrt{\bar{\alpha}_t}\, x_0,\ (1-\bar{\alpha}_t) I\right)$. Para comprobarlo sin usar la fórmula, hacemos el camino largo: tomamos **5,000 copias** de la misma imagen y a cada una le aplicamos 300 veces el paso individual

$$x_t = \sqrt{\alpha_t}\, x_{t-1} + \sqrt{\beta_t}\, \varepsilon_t,$$

con un ruido nuevo e independiente en cada paso. Luego medimos, píxel por píxel, la media y la varianza de las 5,000 copias y las comparamos con $\sqrt{\bar{\alpha}_{300}}\, x_0$ y $1-\bar{\alpha}_{300}$.

La razón de fondo es que **sumar ruidos gaussianos independientes da otro ruido gaussiano**, con varianza igual a la suma de las varianzas. Por eso los 300 ruidos pequeños se pueden reemplazar por uno solo.

Como trabajamos con muestras finitas, la media empírica nunca será idéntica a la teórica. La referencia para decidir si el error es "pequeño" es el **error estándar**: para la media de un píxel es $\sqrt{(1-\bar{\alpha}_t)/N}$, y para la varianza promediada sobre los 784 píxeles es $(1-\bar{\alpha}_t)\sqrt{2/(N-1)}/\sqrt{784}$. Con 784 píxeles, el error máximo de la media debería caer alrededor de 3 o 4 errores estándar.

```
t = 300, alpha_bar_t = 0.396420
media:    error máximo en 784 píxeles = 0.04463 | error estándar = 0.01099 | razón = 4.06
varianza: promedio empírico = 0.60309 | teórica = 0.60358 | diferencia = 0.00049 | error estándar = 0.00043
| razón = 1.13
```

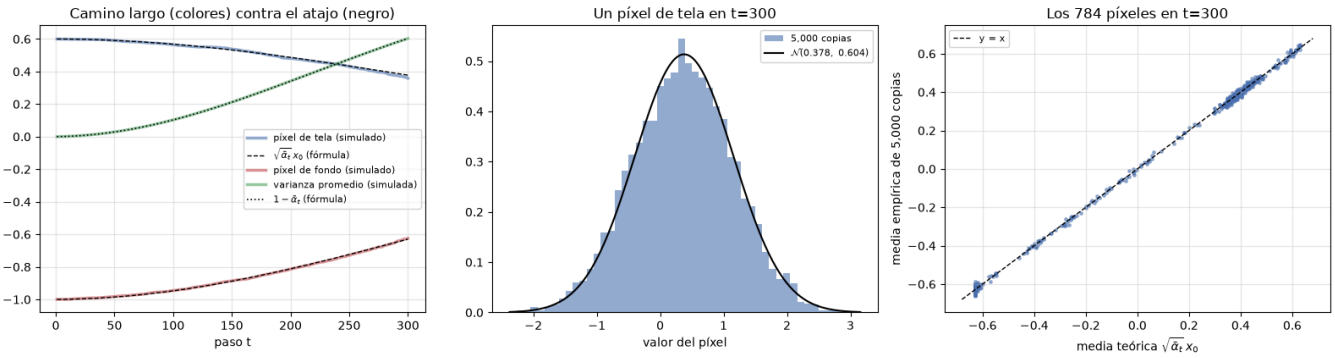



Figura 12. Task 2 · salida de la celda 17 de 02_difusion_forward.ipynb.

En $t = 300$ la fórmula predice $\bar{\alpha}_{300} = 0.3964$, es decir, una varianza de $1-\bar{\alpha}_{300} = 0.6036$.

| Cantidad | Simulado (300 pasos) | Fórmula | Diferencia | Error estándar | Diferencia / error estándar |
|---|---|---|---|---|---|
| Media, error máximo en los 784 píxeles | | | 0.0446 | 0.0110 | 4.06 |
| Varianza promedio | 0.6031 | 0.6036 | 0.0005 | 0.0004 | 1.13 |

La varianza promedio quedó a solo 1.13 errores estándar de la teórica, que es lo que esperaríamos por puro azar. Para la media reportamos el **peor** de los 784 píxeles, y al tomar el máximo de tantos valores lo normal es que alguno se aleje más: con 784 píxeles independientes, la mediana de ese máximo es 3.3 errores estándar y

la probabilidad de ver 4.06 o más es de 3.8 %. El resultado está algo por encima de lo típico, pero es compatible con ruido de muestreo; no indica un error sistemático.

La figura lo confirma de tres formas. A la izquierda, las curvas simuladas paso por paso se montan sobre las de la fórmula durante los 300 pasos, tanto para un píxel de tela como para uno de fondo y para la varianza. En el centro, las 5,000 copias de un píxel forman exactamente la campana $\mathcal{N}(0.378,\ 0.604)$ que predice el atajo. A la derecha, los 784 píxeles caen sobre la recta $y = x$. Concluimos que aplicar 300 ruidos pequeños uno tras otro es equivalente a aplicar un solo ruido con varianza $1 - \bar{\alpha}_t$, porque la suma de gaussianas independientes es otra gaussiana.

## 5. Task 2.3a - Cómo se destruye la imagen

Usamos `q_sample` para ver tres clases con formas muy distintas (Trouser, Sneaker y Bag) en $t \in \{1, 50, 100, 250, 500, 750, 1000\}$. Cada fila usa **un mismo ruido** $\varepsilon$ en todas las columnas; así lo único que cambia de izquierda a derecha es la proporción entre señal y ruido, y no el patrón de manchas.

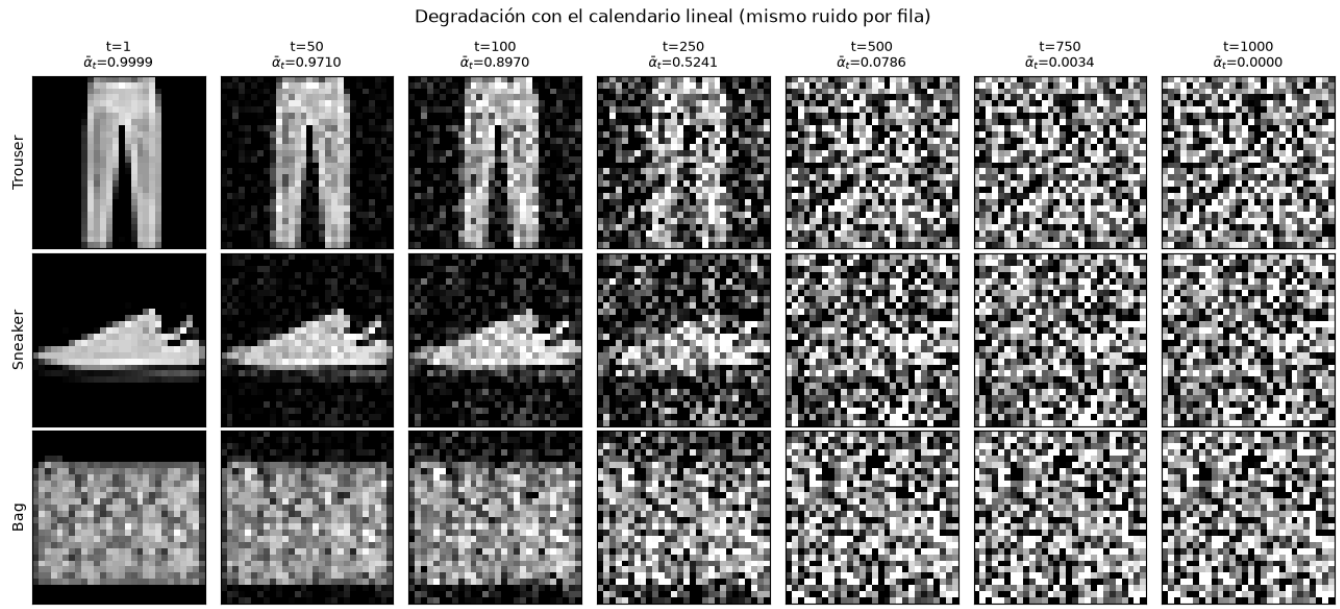



Figura 13. Task 2 · salida de la celda 20 de 02_difusion_forward.ipynb.

Para ver lo mismo de forma global, repetimos el histograma de píxeles del inicio, ahora sobre las 2,000 imágenes ya ensuciadas en cada $t$. La curva negra es la campana $\mathcal{N}(0, 1)$ del ruido puro, a la que el proceso debería llegar.

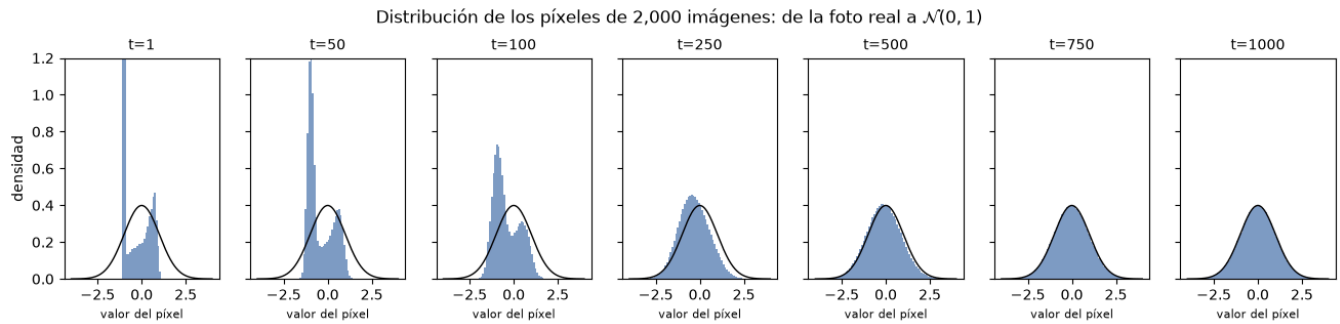



Figura 14. Task 2 · salida de la celda 22 de 02_difusion_forward.ipynb.

En la cuadrícula, hasta $t = 100$ ($\bar{\alpha}_t \approx 0.90$) las tres prendas siguen perfectamente reconocibles: el fondo negro se llena de granitos, pero la forma está intacta. En $t = 250$ ($\bar{\alpha}_t = 0.52$) todavía se adivina la silueta del Sneaker y las dos piernas del Trouser; el Bag es el que más sufre, porque su textura interna se confunde con el ruido. Desde $t = 500$ ($\bar{\alpha}_t = 0.079$) ya no distinguimos ninguna prenda a simple vista, y las columnas $t = 750$ y $t = 1000$ se ven prácticamente iguales entre sí. Es decir, toda la información visual se pierde en la primera mitad del calendario.

Los histogramas cuentan la misma historia para las 2,000 imágenes juntas. En $t = 1$ tenemos la distribución de las fotos: el pico del fondo en $-1$ (recortado en el gráfico porque es mucho más alto) y la tela entre 0.5 y 0.8. Con el tiempo el pico se ensancha, se corre hacia 0 y se funde con el bloque de la tela; en $t = 250$ ya parece una campana, aunque desplazada a la izquierda, y desde $t = 500$ coincide casi por completo con $\mathcal{N}(0, 1)$. Al final, $x_T$ es indistinguible del ruido puro, que es justo el punto de partida que usará el muestreo en la Task 3.

## 6. Task 2.3b - $\bar{\alpha}_t$ y la relación señal a ruido

En $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon$, la potencia de la señal es $\bar{\alpha}_t$ y la del ruido es $1 - \bar{\alpha}_t$ (los coeficientes al cuadrado). Su cociente es la relación señal a ruido:

$$\mathrm{SNR}(t) = \frac{\bar{\alpha}_t}{1 - \bar{\alpha}_t}.$$

Si $\mathrm{SNR} > 1$ en $x_t$ hay más imagen que ruido; si $\mathrm{SNR} < 1$, domina el ruido. Como el SNR va desde casi $10^4$ hasta $10^{-5}$, lo graficamos en escala logarítmica: en escala normal toda la curva después de los primeros pasos se vería pegada al cero.

```
t |  alpha_bar_t |       SNR(t)
-----------------------------------
    1 |     0.999900 |   9.9990e+03
   50 |     0.971016 |   3.3501e+01
  100 |     0.897018 |   8.7104e+00
  250 |     0.524085 |   1.1012e+00
  500 |     0.078587 |   8.5290e-02
  750 |     0.003351 |   3.3618e-03
 1000 |     0.000040 |   4.0360e-05
```

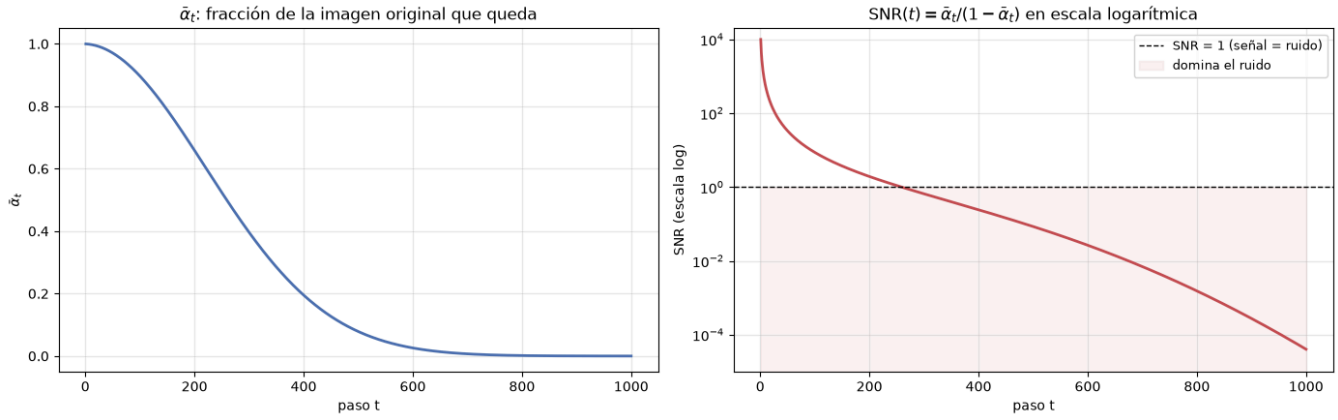



Figura 15. Task 2 · salida de la celda 26 de 02_difusion_forward.ipynb.

La gráfica de $\bar{\alpha}_t$ muestra la caída en escala lineal: desde $t \approx 600$ la curva está pegada al cero y ya no se distingue nada. La del SNR, en escala logarítmica, sí permite ver qué pasa en esa zona. El SNR baja de $\approx 10^4$ en $t = 1$ a $33.5$ en $t = 50$ y $8.7$ en $t = 100$: los primeros pasos destruyen rápido los detalles finos, aunque la imagen siga reconociéndose. Cruza la línea $\mathrm{SNR} = 1$ poco después de $t = 250$ (donde vale $1.10$), y a partir de ahí el ruido domina. Luego sigue bajando de forma casi recta en escala log hasta $4 \times 10^{-5}$ en $t = 1000$.

La conclusión es que con el calendario lineal **cerca de tres cuartas partes de los pasos ocurren con más ruido que señal**, y en los últimos cientos el SNR es tan bajo que $x_t$ es prácticamente ruido puro. En la Task 2.3c ubicamos exactamente esos puntos de cruce.

## 7. Task 2.3c - Dos puntos de referencia

Buscamos dos umbrales sobre el calendario lineal:

- El primer $t$ con $\mathrm{SNR}(t) < 1$, que es cuando el ruido empieza a pesar más que la imagen. Como $\mathrm{SNR} < 1 \iff \bar{\alpha}_t < 1 - \bar{\alpha}_t \iff \bar{\alpha}_t < 0.5$, equivale al primer paso en que queda menos de la mitad de la imagen.
- El primer $t$ con $\bar{\alpha}_t < 0.01$, cuando queda menos del 1 % de la imagen original.

Como $\bar{\alpha}_t$ siempre decrece, basta con encontrar el primer índice que cumple cada condición y sumarle 1 para pasar de índice de PyTorch a paso $t$.

```
Primer t con SNR < 1: t = 260
    t = 259: alpha_bar = 0.500245 | SNR = 1.000979
    t = 260: alpha_bar = 0.497614 | SNR = 0.990501
Primer t con alpha_bar < 0.01: t = 674
    t = 673: alpha_bar = 0.010128 | SNR = 0.010232
    t = 674: alpha_bar = 0.009991 | SNR = 0.010092
Pasos que quedan después de alpha_bar < 0.01: 327 de 1000
```

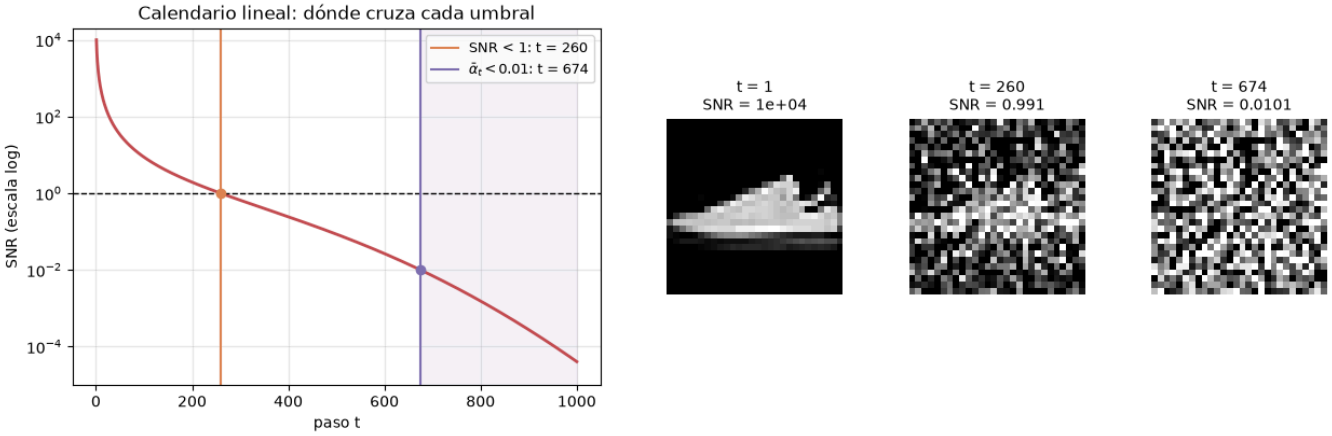



Figura 16. Task 2 · salida de la celda 30 de 02_difusion_forward.ipynb.

| Umbral | Primer $t$ | $\bar{\alpha}_t$ en ese paso | SNR en ese paso |
|---|---|---|---|
| $\mathrm{SNR} < 1$ | **260** | 0.4976 | 0.9905 |
| $\bar{\alpha}_t < 0.01$ | **674** | 0.00999 | 0.0101 |

En el paso anterior a cada umbral la condición todavía no se cumple ($\bar{\alpha}_{259} = 0.5002$ y $\bar{\alpha}_{673} = 0.0101$), así que los dos números son exactos.

El ruido supera a la señal en $t = 260$, apenas a un cuarto del calendario. En la miniatura de ese paso todavía se intuye la silueta del Sneaker, porque un SNR cercano a 1 no significa que la imagen haya desaparecido, solo que pesa lo mismo que el ruido. En cambio, en $t = 674$ ya no queda nada visible. Desde ese paso hasta $t = 1000$ hay **327 pasos** (casi un tercio del calendario) en los que sobrevive menos del 1 % de la imagen. Para el modelo esos pasos aportan muy poco, porque todos sus $x_t$ se parecen al ruido puro; esa es la motivación del calendario de la Task 2.4.

## 8. Task 2.4 - Nuestro calendario: coseno modificado

El calendario lineal destruye la imagen demasiado pronto: el SNR baja de 1 en $t = 260$ y desde $t = 674$ queda menos del 1 % de la imagen. La idea del calendario de coseno (Nichol y Dhariwal, 2021) es definir directamente $\bar{\alpha}_t$ con una curva que baje de forma más pareja:

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \qquad f(t) = \cos^2\left(\frac{t/T + s}{1+s} \cdot \theta_{\max}\right), \qquad s = 0.008.$$

En la versión original $\theta_{\max} = \pi/2$, así que $f(T) = \cos^2(\pi/2) = 0$ y $\bar{\alpha}_T = 0$ exactamente. Eso fuerza $\beta_T = 1 - \bar{\alpha}_T/\bar{\alpha}_{T-1} = 1$, y el artículo lo resuelve recortando los $\beta_t$ a 0.999.

**Nuestra modificación:** en lugar de recortar, detenemos el ángulo un poco antes de $\pi/2$. Elegimos $\theta_{\max}$ (por bisección) para que el calendario termine exactamente en $\bar{\alpha}_T = 5 \times 10^{-4}$, la mitad del límite de $0.001$ que pide la tarea. Así $x_T$ sigue siendo prácticamente ruido puro, ningún $\beta_t$ se dispara a 1 y no hace falta ningún recorte. A partir de $\bar{\alpha}_t$ recuperamos $\beta_t = 1 - \bar{\alpha}_t/\bar{\alpha}_{t-1}$ y $\alpha_t = 1 - \beta_t$, de modo que el calendario tiene la misma forma que el lineal y funciona con `q_sample` sin cambios.

```
theta_max = 1.54844 rad (pi/2 = 1.57080)
alpha_bar_T lineal: 4.036e-05
alpha_bar_T coseno: 5.000e-04  -> cumple < 0.001: True
beta máximo coseno: 0.1244 (sin recorte) | beta_1 = 4.01e-05

    t | a_bar lineal | a_bar coseno |  SNR lineal |  SNR coseno
------------------------------------------------------------------
    1 |      0.99990 |      0.99996 |   9.999e+03 |   2.493e+04
  100 |      0.89702 |      0.97287 |   8.710e+00 |   3.587e+01
  250 |      0.52409 |      0.85111 |   1.101e+00 |   5.716e+00
  500 |      0.07859 |      0.50511 |   8.529e-02 |   1.021e+00
  750 |      0.00335 |      0.15629 |   3.362e-03 |   1.852e-01
  900 |      0.00028 |      0.03065 |   2.753e-04 |   3.162e-02
 1000 |      0.00004 |      0.00050 |   4.036e-05 |   5.003e-04
```

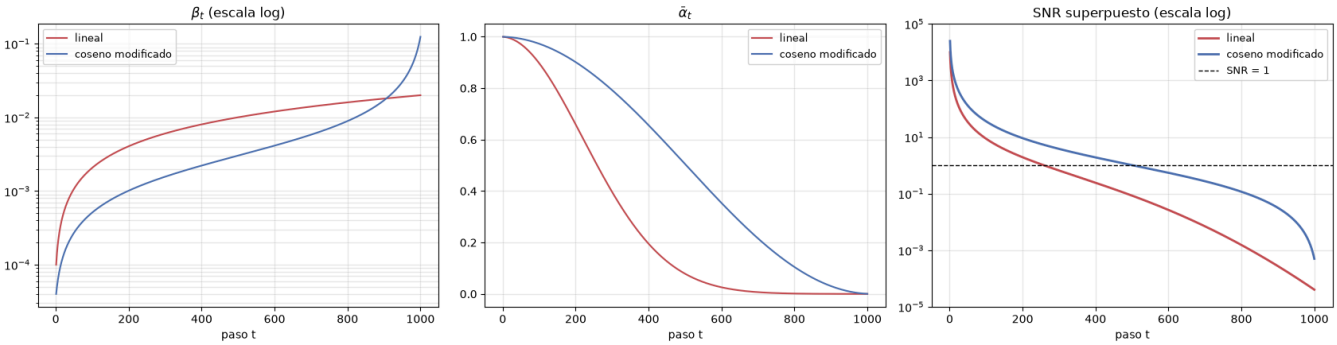



Figura 17. Task 2 · salida de la celda 34 de 02_difusion_forward.ipynb.

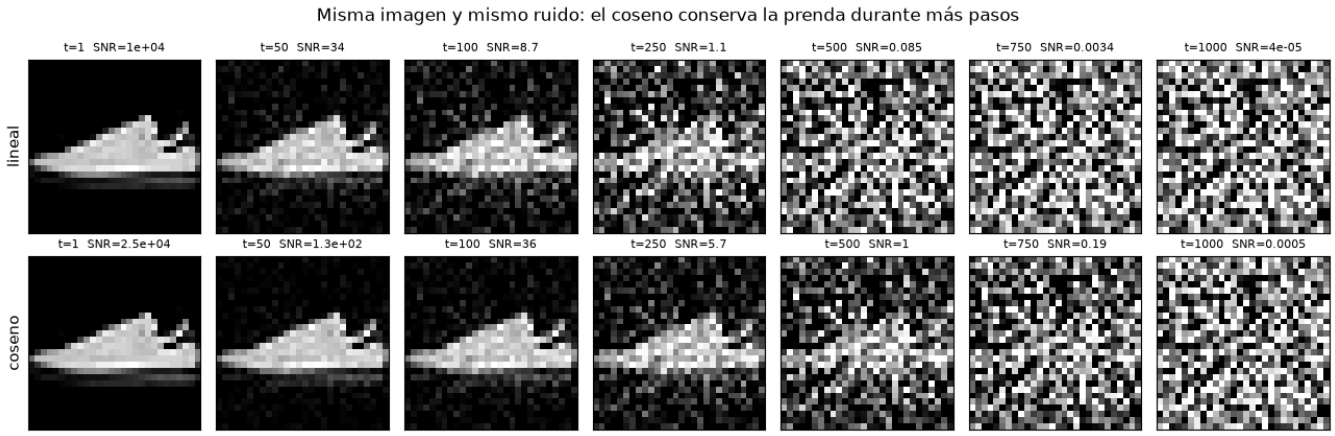



Figura 18. Task 2 · salida de la celda 35 de 02_difusion_forward.ipynb.

El ángulo que encontró la bisección es $\theta_{\max} = 1.5484$ rad, apenas un 1.4 % menor que $\pi/2 = 1.5708$. Con eso el calendario termina en $\bar{\alpha}_T = 5.0 \times 10^{-4}$, que **cumple** $\bar{\alpha}_T < 0.001$. El lineal termina en $\bar{\alpha}_T = 4.0 \times 10^{-5}$, que también lo cumple. El coseno deja un poco más de señal residual al final, pero sigue siendo una mezcla con más de 99.95 % de ruido. El $\beta_t$ más grande es $0.124$ en el último paso, así que no necesitamos ningún recorte.

La diferencia está en cómo se reparten los pasos. En la gráfica de $\beta_t$, el coseno agrega muy poco ruido durante la mayor parte del calendario y concentra los $\beta_t$ grandes en los últimos ~100 pasos. Por eso su $\bar{\alpha}_t$ baja casi en línea recta por el centro en vez de desplomarse al principio: en $t = 500$ el lineal ya solo conserva el 7.9 % de la imagen, mientras que el coseno todavía conserva el 50.5 %. En el SNR superpuesto, el coseno cruza $\mathrm{SNR} = 1$ alrededor de $t = 500$ en lugar de $t = 260$, y se mantiene entre $10^{-2}$ y $10^{2}$ durante casi todo el recorrido. Las miniaturas lo confirman: con el mismo ruido, el Sneaker del coseno se reconoce todavía en $t = 500$, que en el lineal ya es ruido puro. En la Task 2.5b contamos exactamente cuántos pasos útiles gana cada calendario.

## 9. Task 2.5a - $\bar{\alpha}_T$ del calendario lineal a mano

**Paso 1: convertir el producto en una suma.** Tomando logaritmo,

$$\bar{\alpha}_T = \prod_{t=1}^{T}(1-\beta_t) \quad \Longrightarrow \quad \ln \bar{\alpha}_T = \sum_{t=1}^{T} \ln(1-\beta_t).$$

**Paso 2: aproximar cada término.** Por Taylor, $\ln(1-\beta) = -\beta - \frac{\beta^2}{2} - \frac{\beta^3}{3} - \dots$ Como todos los $\beta_t \le 0.02$, el primer término domina:

$$\ln \bar{\alpha}_T \approx -\sum_{t=1}^{T} \beta_t.$$

**Paso 3: la suma aritmética.** Los $\beta_t$ están igualmente espaciados, así que su suma es el número de términos por el promedio del primero y el último:

$$\sum_{t=1}^{T} \beta_t = \frac{T(\beta_1 + \beta_T)}{2} = \frac{1000\,(0.0001 + 0.02)}{2} = 1000 \times 0.01005 = 10.05.$$

**Paso 4: primera aproximación.**

$$\bar{\alpha}_T \approx e^{-10.05} = e^{-10} \cdot e^{-0.05} \approx 4.540 \times 10^{-5} \times 0.9512 \approx 4.32 \times 10^{-5}.$$

**Paso 5: corrección de segundo orden.** El término que despreciamos es $-\frac{1}{2}\sum \beta_t^2$. Para valores igualmente espaciados, $\sum \beta_t^2 \approx T\left(\bar{\beta}^2 + \frac{(\beta_T - \beta_1)^2}{12}\right)$, donde $\bar{\beta} = 0.01005$ es el promedio:

$$\sum_{t=1}^{T} \beta_t^2 \approx 1000\left(0.01005^2 + \frac{0.0199^2}{12}\right) = 1000\,(1.010 \times 10^{-4} + 3.300 \times 10^{-5}) \approx 0.1340.$$

Entonces

$$\bar{\alpha}_T \approx e^{-10.05 - 0.0670} = 4.32 \times 10^{-5} \times e^{-0.067} \approx 4.32 \times 10^{-5} \times 0.9352 \approx 4.04 \times 10^{-5}.$$

La primera aproximación sobrestima $\bar{\alpha}_T$ porque $\ln(1-\beta) < -\beta$: cada factor real quita un poco más que $e^{-\beta}$, y en mil pasos esa diferencia se acumula.

```
suma de beta:   a mano = 10.05000 | código = 10.05000
suma de beta^2: a mano = 0.13400 | código = 0.13407

método                                        |  alpha_bar_T | error relativo
------------------------------------------------------------------------------
1er orden: exp(-suma beta)                    |   4.3186e-05 |        7.006%
2do orden: exp(-suma beta - suma beta^2/2)    |   4.0387e-05 |        0.071%
código: torch.cumprod                         |   4.0358e-05 |        0.000%
```

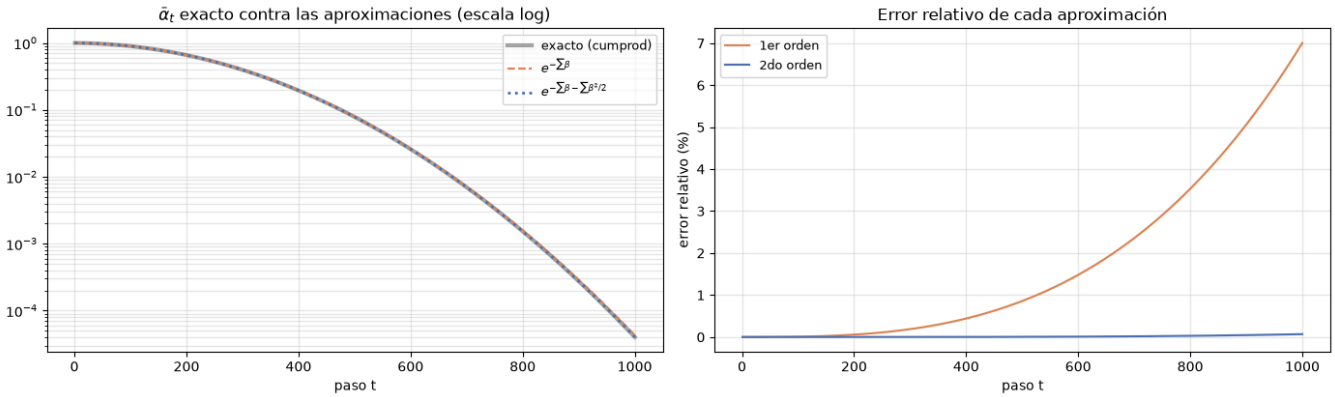



Figura 19. Task 2 · salida de la celda 39 de 02_difusion_forward.ipynb.

| Método | $\bar{\alpha}_T$ | Error relativo |
|---|---|---|
| A mano, 1er orden: $e^{-\sum\beta_t}$ | $4.32 \times 10^{-5}$ | 7.0 % |
| A mano, 2do orden: $e^{-\sum\beta_t-\frac{1}{2}\sum\beta_t^2}$ | $4.04 \times 10^{-5}$ | 0.07 % |
| Código (`torch.cumprod`) | $4.04 \times 10^{-5}$ | referencia |

Las dos sumas a mano coinciden con las del código: $\sum\beta_t = 10.05$ exacto y $\sum\beta_t^2 = 0.1340$ contra $0.13407$. Con solo la suma aritmética ya acertamos el orden de magnitud ($4.3 \times 10^{-5}$) con un error de 7 %, y al agregar la corrección cuadrática el error baja a 0.07 %.

La figura muestra que las tres curvas son indistinguibles a simple vista en escala log, pero el error relativo de primer orden crece con $t$: cada paso aporta un error de aproximadamente $\beta_t^2/2$ en el logaritmo, y esos errores se suman. Como los $\beta_t$ grandes están al final del calendario lineal, el error aparece sobre todo después de $t \approx 500$. La corrección de segundo orden elimina casi todo ese error en todo el recorrido.

## 10. Task 2.5b - ¿Cuántos pasos son útiles?

Durante el entrenamiento, la red recibe $x_t$ con un $t$ elegido al azar y debe predecir el ruido. Si el SNR es muy alto ($> 100$), $x_t$ es casi la imagen limpia y predecir el ruido es trivial, porque basta con ver qué se aparta de una imagen nítida. Si el SNR es muy bajo ($< 0.01$), $x_t$ es casi ruido puro y la mejor predicción posible es "el ruido es $x_t$", sin aprender nada de la imagen. Los pasos donde la red realmente aprende algo son los intermedios:

$$0.01 < \mathrm{SNR}(t) < 100.$$

Contamos cuántos de los 1000 pasos de cada calendario caen en esa franja. Como todos los pasos se muestrean con la misma probabilidad, la fracción de pasos útiles es la fracción del entrenamiento que se aprovecha.

> **Contraste posterior:** la afirmación anterior de que predecir ruido con SNR alto es trivial no coincide con la ejecución de la Task 3. El intervalo 1–100 es el más difícil (MSE=0.2166); la Task 3.5(d) desarrolla esta corrección.

```
lineal: 647 pasos útiles, de t = 28 a t = 674
    muy limpio (SNR > 100)    :   27 pasos (2.7%)
    útil (0.01 < SNR < 100)   :  647 pasos (64.7%)
    casi ruido (SNR < 0.01)   :  326 pasos (32.6%)
coseno: 892 pasos útiles, de t = 58 a t = 949
    muy limpio (SNR > 100)    :   57 pasos (5.7%)
    útil (0.01 < SNR < 100)   :  892 pasos (89.2%)
    casi ruido (SNR < 0.01)   :   51 pasos (5.1%)

El coseno gana 245 pasos útiles (37.9% más que el lineal).
```

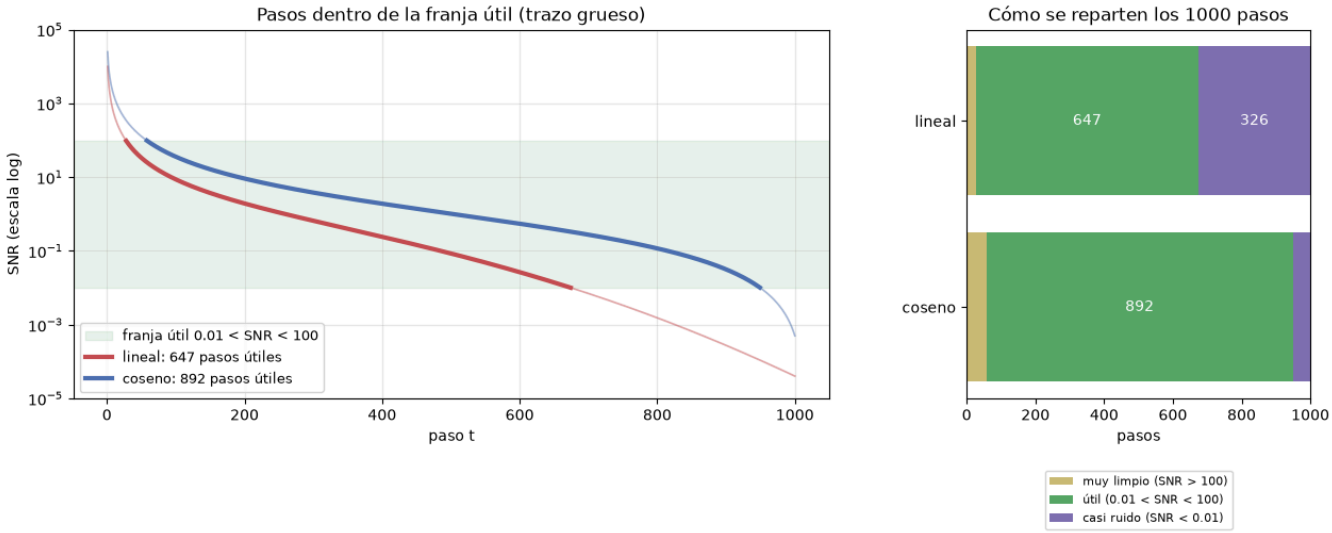



Figura 20. Task 2 · salida de la celda 43 de 02_difusion_forward.ipynb.

| Calendario | SNR > 100 | 0.01 < SNR < 100 | SNR < 0.01 | Franja útil |
|---|---|---|---|---|
| Lineal | 27 | **647** (64.7 %) | 326 | $t = 28$ a $674$ |
| Coseno modificado | 57 | **892** (89.2 %) | 51 | $t = 58$ a $949$ |

**El coseno modificado es más eficiente:** tiene 245 pasos útiles más que el lineal, un 37.9 % más. La diferencia viene casi por completo del final del calendario. El lineal gasta 326 pasos (un tercio del total) con SNR por debajo de 0.01, donde $x_t$ ya es ruido puro; coincide con lo que vimos en la Task 2.3c, porque $\mathrm{SNR} < 0.01$ equivale a $\bar{\alpha}_t < 0.0099$. El coseno reduce esa zona a solo 51 pasos.

El coseno sí cede un poco al principio: tiene 57 pasos con SNR mayor que 100, contra 27 del lineal, porque agrega el ruido más despacio en los primeros pasos. Pero ese costo es de 30 pasos frente a los 275 que recupera al final, así que el balance es claramente favorable. En la gráfica de la izquierda se ve por qué: la curva azul recorre la franja verde de forma pareja durante casi todo el calendario, mientras que la roja la atraviesa rápido y sale de ella en $t = 674$.

## 11. Task 2.5c - La varianza de $x_t$

**Demostración.** Partimos de la forma cerrada $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon$, donde $\varepsilon \sim \mathcal{N}(0, I)$ se genera **independiente** de la imagen $x_0$. Para dos variables aleatorias $X$, $Y$ y constantes $a$, $b$,

$$\mathrm{Var}(aX + bY) = a^2\,\mathrm{Var}(X) + b^2\,\mathrm{Var}(Y) + 2ab\,\mathrm{Cov}(X, Y).$$

Con $a = \sqrt{\bar{\alpha}_t}$, $b = \sqrt{1 - \bar{\alpha}_t}$, $X = x_0$ y $Y = \varepsilon$:

- $a^2 = \bar{\alpha}_t$ y $b^2 = 1 - \bar{\alpha}_t$;
- $\mathrm{Var}(\varepsilon) = 1$, porque es ruido normal estándar;
- $\mathrm{Cov}(x_0, \varepsilon) = 0$, porque son independientes.

Por lo tanto

$$\boxed{\mathrm{Var}(x_t) = \bar{\alpha}_t\,\mathrm{Var}(x_0) + (1 - \bar{\alpha}_t)}. \qquad \blacksquare$$

**Qué significa.** La varianza de $x_t$ es un promedio ponderado entre la de los datos y la del ruido, con los mismos pesos $\bar{\alpha}_t$ y $1 - \bar{\alpha}_t$ que suman 1. En $t = 0$ vale $\mathrm{Var}(x_0)$ y, como $\bar{\alpha}_T \approx 0$, al final vale $\approx 1$, que es justo la varianza de $\mathcal{N}(0, I)$. Con la media pasa lo mismo: $\mathbb{E}[x_t] = \sqrt{\bar{\alpha}_t}\,\mathbb{E}[x_0] \to 0$. Así, el final del proceso encaja con la distribución desde la que arranca el muestreo.

Para comprobarla tomamos 10,000 imágenes de entrenamiento, les aplicamos `q_sample` con ruido nuevo y medimos la varianza de todos sus píxeles.

```
10000 imágenes | Var(x_0) = 0.50131 | E[x_0] = -0.42738

calendario |     t | alpha_bar_t | Var teórica | Var empírica | diferencia |   E[x_t]
-------------------------------------------------------------------------------------
    lineal |     1 |    0.999900 |     0.50136 |      0.50136 |   -0.00000 |  -0.4274
```

```
lineal |   250 |    0.524085 |     0.73865 |      0.73852 |   -0.00012 |  -0.3095
    lineal |   500 |    0.078587 |     0.96081 |      0.96077 |   -0.00004 |  -0.1198
```

```
lineal |  1000 |    0.000040 |     0.99998 |      1.00029 |   +0.00031 |  -0.0030
    coseno |     1 |    0.999960 |     0.50133 |      0.50133 |   -0.00000 |  -0.4274
```

```
coseno |   250 |    0.851108 |     0.57557 |      0.57553 |   -0.00004 |  -0.3942
    coseno |   500 |    0.505112 |     0.74811 |      0.74826 |   +0.00015 |  -0.3041
```

```
coseno |  1000 |    0.000500 |     0.99975 |      0.99940 |   -0.00035 |  -0.0102
```

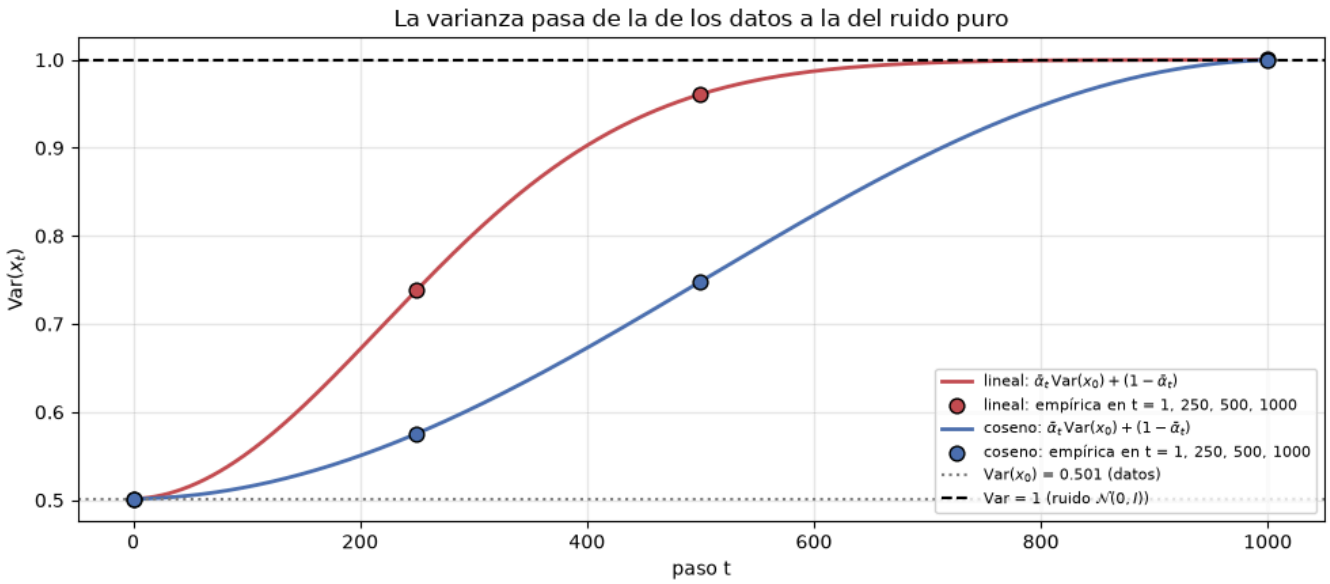



Figura 21. Task 2 · salida de la celda 47 de 02_difusion_forward.ipynb.

Con 10,000 imágenes medimos $\mathbf{Var}(x_0) = 0.5013$. La comprobación en los cuatro pasos pedidos, con el calendario lineal:

| $t$ | $\bar{\alpha}_t$ | Var teórica | Var empírica | Diferencia |
|---|---|---|---|---|
| 1 | 0.99990 | 0.50136 | 0.50136 | 0.00000 |
| 250 | 0.52409 | 0.73865 | 0.73852 | −0.00012 |
| 500 | 0.07859 | 0.96081 | 0.96077 | −0.00004 |
| 1000 | 0.00004 | 0.99998 | 1.00029 | +0.00031 |

Todas las diferencias son menores que $4 \times 10^{-4}$, del tamaño esperado por usar una muestra finita. Con el coseno modificado ocurre lo mismo (diferencias de $3.5 \times 10^{-4}$ como máximo), solo que la varianza sube más despacio: en $t = 500$ vale $0.748$ en lugar de $0.961$, otra forma de ver que conserva la imagen por más tiempo.

En $t = 1000$ los dos calendarios llegan a $\mathbf{Var}(x_T) \approx 1$ y $\mathbb{E}[x_T] \approx 0$ ($-0.003$ en el lineal y $-0.010$ en el coseno, que deja un poco más de señal). Es decir, $x_T$ tiene la media y la varianza de $\mathcal{N}(0, I)$: el final del proceso forward encaja con el ruido puro desde el que el modelo empezará a generar en la Task 3.

# Laboratorio 9 - Task 3: difusión condicional con guía sin clasificador

En esta parte entrenamos una U-Net pequeña que predice el ruido $\epsilon$ a partir de $x_t$, del instante $t$ y de una clase $c$. Con ella generamos imágenes mediante el muestreo ancestral y medimos el efecto de la guía sin clasificador sobre la fidelidad y la diversidad.

Todo el código reutilizable vive en `src/difusion.py` (con pruebas en `tests/test_difusion.py`). El calendario lineal y la función `q_sample` son los mismos de la Task 2.1.

## 1. Datos

Seguimos el mismo corte de la Task 1.1: las primeras 55 000 imágenes de entrenamiento entrenan y las últimas 5 000 validan. Los píxeles pasan de $[0, 255]$ a $[-1, 1]$ con $x \mapsto 2\,x/255 - 1$, que es la misma transformación que `ToTensor` seguido de `Normalize(0.5, 0.5)` en la Task 2.

```
train / val: 55000 5000
rango de píxeles: [-1.0, 1.0]
```

## 2. Task 3.1 - U-Net condicional

La red tiene dos niveles de resolución. A $28 \times 28$ trabaja con 32 canales; una convolución con stride 2 baja a $14 \times 14$ con 64 canales, donde está el cuello de botella. El camino de subida concatena las características del mismo nivel (conexiones de salto) y termina con una convolución a un canal.

El instante entra con el embedding sinusoidal de la Semana 6,

$$\mathrm{PE}(t, 2i) = \sin\!\left(\frac{t}{10000^{2i/d}}\right), \qquad \mathrm{PE}(t, 2i+1) = \cos\!\left(\frac{t}{10000^{2i/d}}\right),$$

seguido de un MLP. La clase entra con `nn.Embedding(11, d)`, donde la clase 10 es la condición nula. Ambos vectores se suman y cada bloque proyecta esa suma a sus canales y la añade a todos los píxeles por broadcasting.

```
forma de entrada: (4, 1, 28, 28) | forma de salida: (4, 1, 28, 28)
Parámetros totales: 395,009
```

## 3. Task 3.2 - Entrenamiento

En cada iteración, para cada imagen del lote, sorteamos $t \sim \mathcal{U}\{1, \dots, T\}$ y $\epsilon \sim \mathcal{N}(0, I)$, construimos $x_t = \sqrt{\bar{\alpha}_t}\,x_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon$ con `q_sample`, reemplazamos la clase por la nula con probabilidad 0.1 y minimizamos

$$\mathcal{L} = \|\epsilon - \epsilon_\theta(x_t, t, c)\|^2.$$

Cada época guarda un checkpoint, así que si la ejecución se corta se reanuda desde la última época completa.

**Configuración:** 30 épocas, batch_size=128, lr=0.0002 y p_uncond=0.1; NVIDIA GeForce RTX 3050 6GB Laptop GPU.

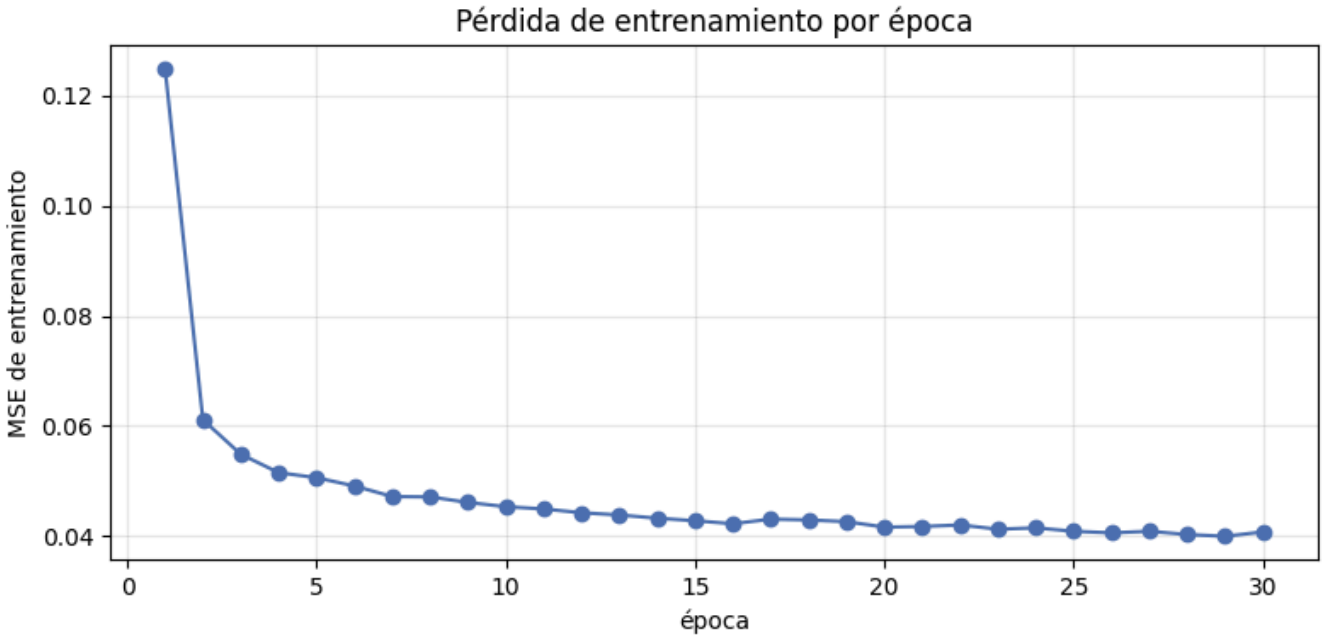



Figura 22. Task 3 · salida de la celda 9 de 03_difusion_condicional.ipynb.

```
Pérdida en la época 1: 0.1250 | en la época 30: 0.0408
```

Al terminar evaluamos sobre las 5 000 imágenes de validación la pérdida media en cada uno de los 10 intervalos de $t$ de 100 pasos. Dentro de cada intervalo $t$ se sortea de forma uniforme, la clase es la real y el ruido sale de un generador fijo para que todos los intervalos se comparen con los mismos sorteos.

```
intervalo de t |  MSE val
---------------------------
          1-100 |   0.2166
        101-200 |   0.0828
        201-300 |   0.0472
        301-400 |   0.0279
        401-500 |   0.0156
        501-600 |   0.0077
        601-700 |   0.0032
        701-800 |   0.0013
        801-900 |   0.0007
       901-1000 |   0.0007
```

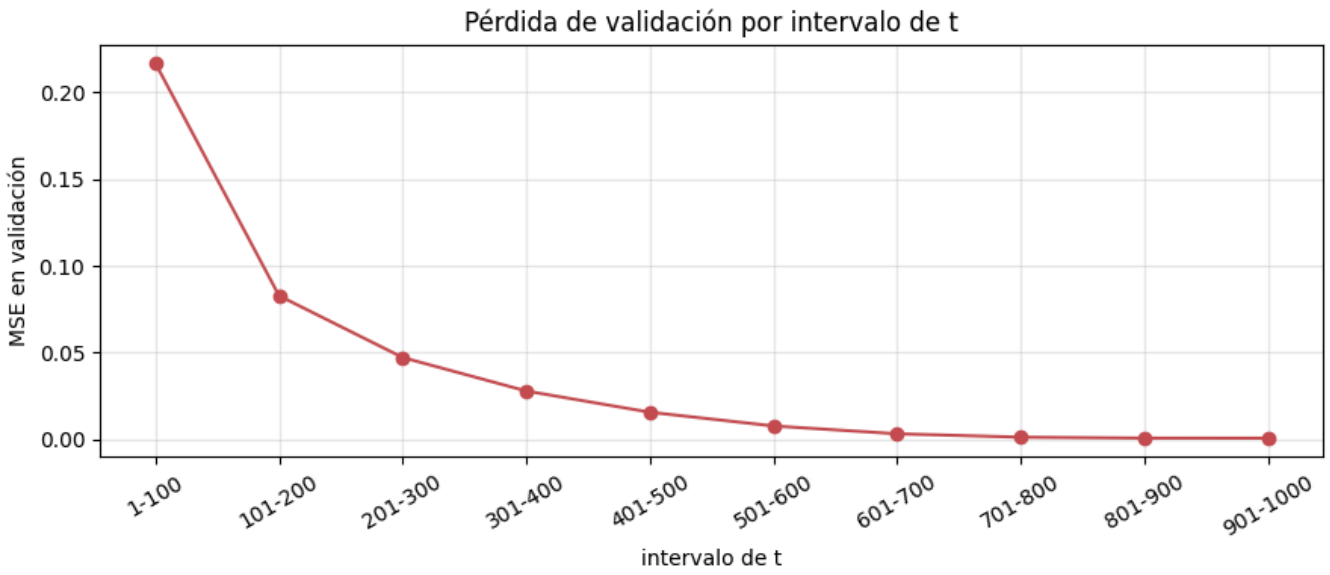



Figura 23. Task 3 · salida de la celda 11 de 03_difusion_condicional.ipynb.

## 4. Task 3.3 - Muestreo ancestral

Partimos de $x_T \sim \mathcal{N}(0, I)$ y para $t = T, \ldots, 1$ aplicamos

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \hat{\epsilon} \right) + \sigma_t z, \qquad \sigma_t^2 = \beta_t,$$

con $z = 0$ en el último paso. Aquí usamos solo la predicción condicional ($w = 1$) y recortamos el resultado a $[-1, 1]$.

```
forma: (100, 1, 28, 28) | rango: [-1.00, 1.00]
```




Figura 24. Task 3 · salida de la celda 13 de 03_difusion_condicional.ipynb.

## 5. Task 3.4 - Guía sin clasificador

La predicción guiada combina la predicción condicional y la nula:

$$\tilde{\epsilon} = \epsilon_\theta(x_t, t, \varnothing) + w\left(\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \varnothing)\right).$$

Con $w = 1$ la fórmula se reduce exactamente a $\epsilon_\theta(x_t, t, c)$, así que en ese caso hacemos una sola pasada por la red (1 000 evaluaciones por imagen). Con $w > 1$ hacen falta dos pasadas por paso (2 000 evaluaciones), que se calculan juntas en un mismo lote.

### 5.1 Clasificador

Para medir la fidelidad entrenamos una CNN sencilla sobre las imágenes reales de entrenamiento, en la misma escala $[-1, 1]$, hasta superar 88 % de exactitud en validación.

```
época  1 | pérdida 0.4832 | exactitud val 0.8772
```

```
época  2 | pérdida 0.2956 | exactitud val 0.8964
Exactitud final en validación: 89.64%
```

### 5.2 Generación con $w \in \{1, 3, 7\}$

Generamos 50 imágenes por clase (500 en total) para cada $w$ y medimos el tiempo de generación. En GPU se sincroniza antes y después para que el tiempo incluya todo el trabajo pendiente.

```
w = 1: 500 imágenes en 56.7 s -> 11.34 s por cada 100
```

```
w = 3: 500 imágenes en 112.8 s -> 22.55 s por cada 100
```

```
w = 7: 500 imágenes en 108.9 s -> 21.78 s por cada 100
```

### 5.3 Fidelidad y diversidad

La **fidelidad** es la fracción de imágenes que el clasificador asigna a la clase pedida. La **diversidad** es la distancia euclidiana media entre pares distintos de imágenes de la misma clase (sobre los 784 píxeles en $[-1, 1]$), promediada sobre las 10 clases. Como referencia usamos las primeras 50 imágenes reales de cada clase del conjunto de validación.

```
w | fidelidad | diversidad | s / 100 imgs
-------------------------------------------------
     1 |     0.838 |     16.182 |        11.34
     3 |     0.986 |     14.000 |        22.55
     7 |     0.998 |     12.942 |        21.78
  real |     0.880 |     17.675 |           -
```

```
clase |     w=1 |     w=3 |     w=7 |    real | caída 1->7
-----------------------------------------------------------------
 T-shirt/top |  15.065 |  11.681 |  11.226 |  18.741 |      3.840
     Trouser |  13.888 |   9.881 |   9.448 |  14.471 |      4.440
    Pullover |  16.041 |  14.554 |  11.574 |  18.559 |      4.467
       Dress |  16.231 |  14.193 |  13.211 |  16.112 |      3.019
        Coat |  15.931 |  13.487 |  12.131 |  16.895 |      3.800
      Sandal |  16.624 |  15.551 |  17.323 |  18.917 |     -0.699
       Shirt |  17.409 |  16.345 |  14.412 |  19.994 |      2.997
     Sneaker |  14.506 |  11.651 |  11.344 |  12.744 |      3.162
         Bag |  19.755 |  19.459 |  18.740 |  22.641 |      1.014
  Ankle boot |  16.366 |  13.196 |  10.011 |  17.678 |      6.355

Clase con mayor caída de diversidad entre w=1 y w=7: Ankle boot (16.366 -> 10.011)
```

### 5.4 Muestras de una clase para cada $w$

Mostramos 10 muestras de la clase cuya diversidad cae más entre $w = 1$ y $w = 7$.




Figura 25. Task 3 · salida de la celda 22 de 03_difusion_condicional.ipynb.

### 5.5 Matriz de confusión con $w = 1$

Filas = clase pedida, columnas = clase que asigna el clasificador. El par más confundido se toma como el de mayor suma de errores en ambas direcciones.

```
filas = clase pedida, columnas = clase predicha
tensor([[42,  0,  1,  0,  0,  0,  7,  0,  0,  0],
        [ 0, 47,  0,  3,  0,  0,  0,  0,  0,  0],
        [ 0,  0, 33,  0,  9,  0,  8,  0,  0,  0],
        [ 1,  1,  0, 45,  1,  0,  2,  0,  0,  0],
        [ 0,  0,  1,  2, 42,  0,  5,  0,  0,  0],
        [ 0,  0,  0,  0,  0, 47,  0,  2,  0,  1],
        [ 1,  0,  7,  2,  8,  0, 32,  0,  0,  0],
        [ 0,  0,  0,  0,  0,  5,  0, 40,  2,  3],
        [ 0,  0,  1,  0,  0,  2,  0,  0, 46,  1],
        [ 0,  0,  0,  0,  0,  1,  0,  4,  0, 45]])

Par más confundido: Pullover <-> Shirt (8 de Pullover como Shirt, 7 de Shirt como Pullover)
```

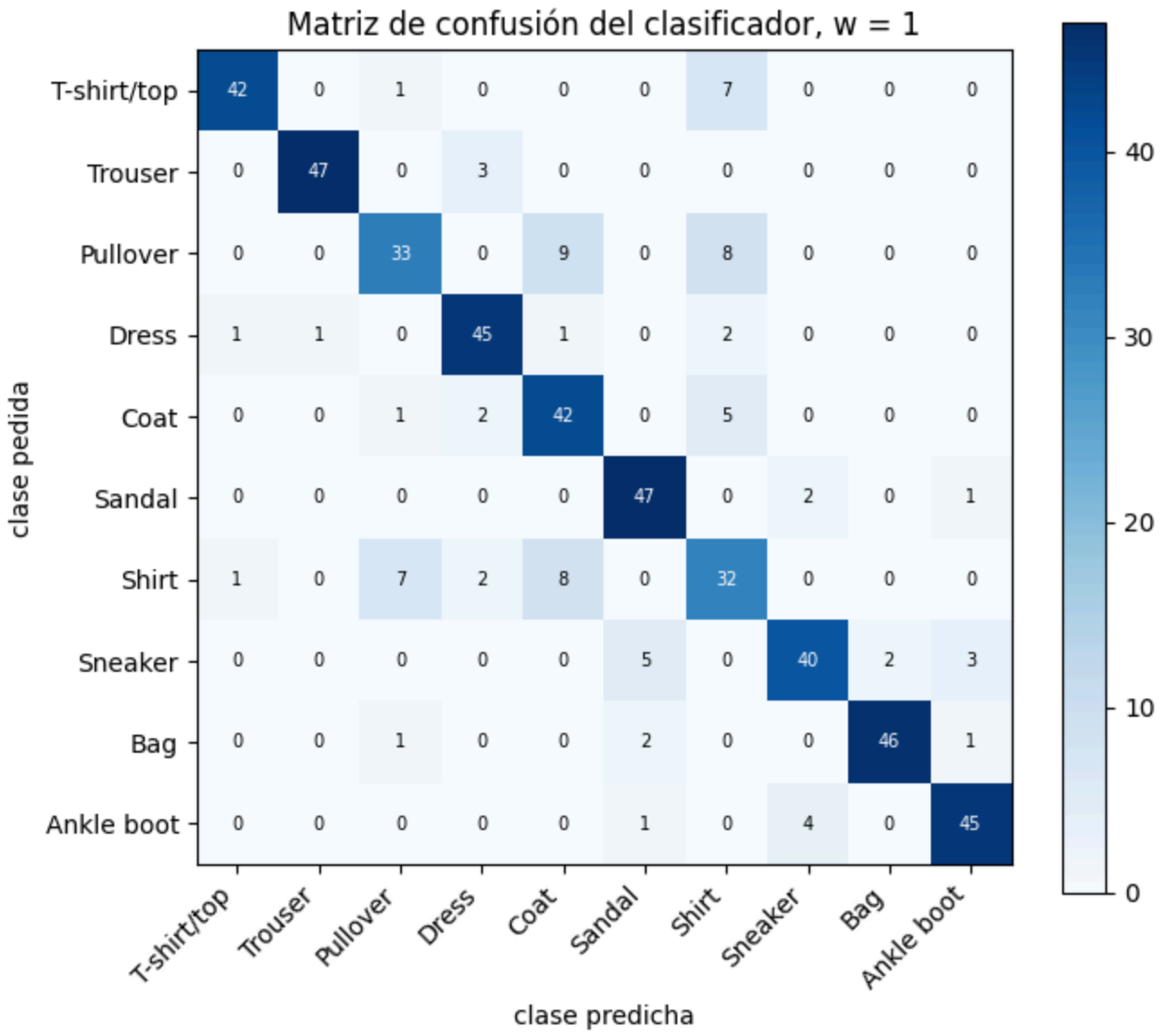



Figura 26. Task 3 · salida de la celda 24 de 03_difusion_condicional.ipynb.




Figura 27. Task 3 · salida de la celda 25 de 03_difusion_condicional.ipynb.

```
Pullover -> Shirt: 8 imágenes
Shirt -> Pullover: 7 imágenes
```

Para separar un defecto del generador de uno del clasificador o de las clases, medimos cuánto confunde el clasificador ese mismo par en las 5 000 imágenes **reales** de validación.

```
real Pullover como Shirt: 49 de 490 (10.0%)
real Shirt como Pullover: 19 de 467 (4.1%)
generadas w=1 Pullover como Shirt: 8 de 50 (16.0%)
generadas w=1 Shirt como Pullover: 7 de 50 (14.0%)

exactitud por clase en reales:
   T-shirt/top: 81.8%
       Trouser: 98.2%
      Pullover: 75.3%
         Dress: 93.7%
          Coat: 88.0%
        Sandal: 97.6%
         Shirt: 70.0%
       Sneaker: 96.4%
           Bag: 99.0%
    Ankle boot: 95.2%
```

# 7. Task 3.5 - respuestas

### a. Fidelidad y diversidad al aumentar $w$

Al subir $w$ la fidelidad crece de 0.838 con $w = 1$ a 0.986 con $w = 3$ y 0.998 con $w = 7$, por encima de la fidelidad de las imágenes reales (0.880). La diversidad cae de 16.182 a 14.000 y 12.942, mientras la de las reales es 17.675. Más guía produce imágenes más fáciles de reconocer y más parecidas entre sí.

La clase con mayor caída es Ankle boot, de 16.366 a 10.011, una caída de 6.355. La hipótesis es que la guía empuja cada muestra hacia la zona donde la clase es inequívoca, y en Ankle boot esa zona es estrecha porque la clase limita con Sneaker y Sandal. Con $w = 1$ la matriz de confusión registra 4 Ankle boot generadas como Sneaker y 1 como Sandal. En la figura de 10 muestras, con $w = 1$ aparecen botas bajas con cordones y suela plana que se parecen a un tenis, y con $w = 7$ casi todas son la misma bota alta de tacón cuadrado. La guía elimina justo las variantes cercanas a Sneaker, que eran las que aportaban variedad.

### b. Matriz de confusión con $w = 1$

El par más confundido es Pullover y Shirt, con 8 Pullover generadas clasificadas como Shirt y 7 Shirt clasificadas como Pullover. En las imágenes reales de validación el clasificador también los confunde, con 10.0 % de Pullover como Shirt y 4.1 % de Shirt como Pullover, y son sus dos clases de menor exactitud (75.3 % y 70.0 %). En clases de silueta inconfundible como Trouser y Bag llega a 98.2 % y 99.0 %.

La confusión viene principalmente de las propias clases. En la figura del par, las reales de ambas clases comparten la silueta de manga larga y se distinguen sobre todo por el cuello y la línea de botones de la camisa. El modelo generativo agrava el problema, sobre todo de Shirt hacia Pullover, que pasa de 4.1 % en reales a 14.0 % en generadas. Las camisas generadas mal clasificadas no muestran un cuello ni botones visibles y quedan como un suéter liso. El clasificador aporta error, pero ese error nace de la semejanza entre las clases y no de un fallo general, porque su exactitud global es 89.64 %.

### c. La guía como muestreo de $\tilde{p}(x \mid c) \propto p(x)\, p(c \mid x)^w$

Al tomar el logaritmo y el gradiente respecto a $x$, la constante de normalización desaparece porque no depende de $x$.

$$\nabla_x \log \tilde{p}(x \mid c) = \nabla_x \log p(x) + w\, \nabla_x \log p(c \mid x)$$

Por la regla de Bayes $p(c \mid x) = p(x \mid c)\, p(c) / p(x)$, y como $p(c)$ no depende de $x$,

$$\nabla_x \log p(c \mid x) = \nabla_x \log p(x \mid c) - \nabla_x \log p(x).$$

Al sustituir,

$$\nabla_x \log \tilde{p}(x \mid c) = \nabla_x \log p(x) + w\big(\nabla_x \log p(x \mid c) - \nabla_x \log p(x)\big).$$

Con $s \approx -\epsilon_\theta / \sqrt{1 - \bar{\alpha}_t}$ el score incondicional es $-\epsilon_\theta(x_t, t, \varnothing)/\sqrt{1 - \bar{\alpha}_t}$ y el condicional es $-\epsilon_\theta(x_t, t, c)/\sqrt{1 - \bar{\alpha}_t}$. Multiplicando toda la igualdad por $-\sqrt{1 - \bar{\alpha}_t}$ queda

$$\tilde{\epsilon} = \epsilon_\theta(x_t, t, \varnothing) + w\big(\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \varnothing)\big),$$

que es la predicción guiada. Con $w = 1$ se recupera $p(x)\,p(c \mid x) \propto p(x \mid c)$, el modelo condicional puro. Con $w > 1$ la probabilidad $p(c \mid x)$, que es menor que 1, se eleva a una potencia mayor. Eso castiga más las regiones donde la clase es dudosa y concentra la masa donde $p(c \mid x)$ es cercana a 1. La distribución se vuelve más estrecha, y por eso la diversidad cae de 16.182 con $w = 1$ a 12.942 con $w = 7$ mientras la fidelidad llega a 0.998.

### d. Pérdida por intervalo de $t$

La pérdida es mayor en el intervalo 1-100 (0.2166) y menor en 801-900 y 901-1000 (0.0007), y disminuye de un intervalo al siguiente hasta estabilizarse en 0.0007. En $t$ pequeño el SNR es muy alto, $x_t$ es casi $x_0$ y el ruido es una perturbación diminuta que hay que separar de los detalles finos de la prenda. En $t$ grande el SNR es menor que 0.01, $x_t \approx \epsilon$ y predecir $\epsilon$ es casi copiar la entrada.

En la Task 2.5b el calendario lineal tiene 326 pasos con SNR menor que 0.01 (de $t = 675$ en adelante), 27 con SNR mayor que 100 y 647 útiles entre $t = 28$ y $t = 674$. El comportamiento coincide en el régimen de SNR bajo, porque los intervalos de 701 a 1000 tienen pérdida de 0.0013 o menos y aportan muy poca señal de aprendizaje. No coincide en el régimen de SNR alto. La Task 2.5b suponía que predecir el ruido con SNR mayor que 100 era trivial, pero el intervalo 1-100 es el más difícil, con 0.2166 frente a 0.0007 en el intervalo 901-1000. La franja útil sí coincide, porque los intervalos de 101 a 700 tienen pérdidas intermedias, entre 0.0828 y 0.0032.

# Laboratorio 9 — Task 4: comparación generativa y pass@k

Reutilizamos los pesos y las 500 muestras con $w = 1$ de la Task 3.4. Como no se incluyeron pesos del VAE en el repositorio, repetimos únicamente el entrenamiento de $\beta = 1$: misma arquitectura, semilla 3045, 20 épocas, Adam con lr=0.001 y lotes de 128. Es una nueva ejecución, no los pesos originales de la Task 1. Las definiciones del VAE y su entrenamiento se copian directamente del notebook 01 al construir este documento.

## Task 4.1(a) — Nitidez de 500 imágenes por familia

Medimos $\mathbf{media}\,|x_{i,j+1} - x_{i,j}|$ con todos los píxeles en $[0, 1]$. Las muestras de difusión guardadas están cuantizadas a uint8: dividimos entre 255, **no** comparamos directamente su escala con la del VAE. La cuantización introduce como máximo $1/255$ de diferencia en esta métrica respecto a las muestras flotantes originales. Usamos las primeras 500 imágenes de validación como referencia real y 500 muestras del prior del VAE.

```
Reales          | n=500 | nitidez=0.088579
VAE β=1         | n=500 | nitidez=0.044968
Difusión w=1    | n=500 | nitidez=0.089737
```




Figura 28. Task 4 · salida de la celda 8 de 04_comparacion_passk.ipynb.

## Task 4.1(b) — Tiempo de generación de 100 imágenes

«Sin guía» significa aquí $w = 1$, la predicción condicional pura de la Task 3.3, sin una segunda evaluación incondicional. Ambos modelos usan CPU, float32, cuatro hilos y un lote de 100. Medimos la mediana de tres repeticiones después de un calentamiento. Incluimos el sorteo del ruido y el muestreo; excluimos carga de pesos, entrenamiento, gráficos y escritura. Son tiempos locales, no los tiempos de la GPU usados en la Task 3.

El VAE evalúa su decoder una vez por lote (una vez por imagen). La difusión evalúa la U-Net 1000 veces por lote (1000 veces por imagen); no son 1000 llamadas adicionales por cada imagen porque las 100 se procesan juntas.

```
VAE: repetición 1, 0.000261 s
```

```
VAE: repetición 2, 0.000319 s
```

```
VAE: repetición 3, 0.000291 s
```

```
Difusión: repetición 1, 56.237384 s
```

```
Difusión: repetición 2, 56.175745 s
```

```
Difusión: repetición 3, 56.362619 s
```

```
Razón difusión/VAE: 193006.9 veces
```

## Task 4.1(c) — Compensaciones observadas

El VAE obtuvo nitidez **0.044968**, frente a **0.089737** de difusión y **0.088579** de las imágenes reales. La difusión conserva más variación local; esta métrica también puede aumentar por ruido y no demuestra por sí sola calidad semántica. En la figura, el VAE suaviza contornos y texturas, mientras la difusión conserva más detalle. El VAE tarda **0.000291 s** por 100 imágenes y la difusión **56.237 s**: **193006.9 veces** más tiempo, con 1 frente a 1000 evaluaciones por lote.

El VAE prioriza velocidad a costa de nitidez: comprime a dos dimensiones y su reconstrucción cuadrática favorece promedios. Su pérdida de validación pasó de **33.2291** a **28.3794**. En la Task 3 la pérdida de difusión pasó de **0.1250** a **0.0408** en 30 épocas. Ambas corridas convergieron sin un entrenamiento adversarial, pero sus pérdidas tienen escalas distintas y una sola semilla no prueba estabilidad general.

Para acelerar difusión implementaría un muestreador de pasos reducidos, por ejemplo DDIM con 50 pasos, y volvería a medir nitidez y fidelidad. Eso reduciría las evaluaciones teóricas de 1000 a 50; no basta con truncar los últimos pasos del muestreador ancestral ni implica una aceleración medida de 20 veces.

## Task 4.2(a) — Estimador insesgado

El fracaso significa escoger $k$ soluciones entre las $n - c$ incorrectas, sin reemplazo. Por eso $\text{pass@}k = 1 - \binom{n-c}{k} / \binom{n}{k}$. Usamos un producto de cocientes en float64, sin factoriales grandes.

Para P2, $n = 20$, $c = 2$, $k = 5$:

$$\binom{18}{5} = 8568, \qquad \binom{20}{5} = 15504,$$

$$\text{pass@}5 = 1 - \frac{8568}{15504} = \frac{17}{38} = 0.4473684211.$$

```
Problema | c | pass@1   | pass@5   | pass@10  | ingenuo@5 | diferencia@5
P1       |  0| 0.000000 | 0.000000 | 0.000000 | 0.000000  | 0.000000
P2       |  2| 0.100000 | 0.447368 | 0.763158 | 0.409510  | 0.037858
P3       |  5| 0.250000 | 0.806308 | 0.983746 | 0.762695  | 0.043613
P4       |  9| 0.450000 | 0.970201 | 0.999940 | 0.949672  | 0.020530
P5       | 14| 0.700000 | 0.999613 | 1.000000 | 0.997570  | 0.002043
P6       | 20| 1.000000 | 1.000000 | 1.000000 | 1.000000  | 0.000000
Promedios: pass@1=0.416667, pass@5=0.703915, pass@10=0.791141
P2: menor k con pass@k > 0.9: 14
P2: pass@13=0.889474, pass@14=0.921053
```

## Task 4.2(b) — Umbral para P2

Al cancelar los factoriales, para $1 \leq k \leq 18$:

$$\frac{\binom{18}{k}}{\binom{20}{k}} = \frac{18!\,(20-k)!}{20!\,(18-k)!} = \frac{(20-k)(19-k)}{380}.$$

Para $k = 19, 20$ no pueden elegirse tantas soluciones incorrectas y pass@k vale 1; la misma expresión también da 1. Exigir pass@k > 0.9 equivale a $(20-k)(19-k) < 38$. Con $k = 13$ el producto es 42 y pass@13 = 0.889474; con $k = 14$ es 30 y pass@14 = 0.921053. Como pass@k es monótono, **14 es el mínimo**.

## Task 4.2(c) — Comparación con la versión ingenua

Si $n - c \geq k$, cada factor no negativo cumple

$$\frac{n-c-i}{n-i} = 1 - \frac{c}{n-i} \leq 1 - \frac{c}{n}.$$

Al multiplicar para $i = 0, \ldots, k-1$ y restar de 1:

$$1 - \prod_{i} \frac{n-c-i}{n-i} \geq 1 - (1 - c/n)^k.$$

Si $n - c < k$, el estimador insesgado vale 1 y la desigualdad también se cumple. En la tabla, P2 da 0.447368 frente a 0.409510; P3 da 0.806308 frente a 0.762695. La versión ingenua nunca excede a la insesgada. Modela intentos independientes con reemplazo y trata c/n como una probabilidad exacta; no es el estimador insesgado a partir de las 20 soluciones observadas.

## Task 4.2(d) — Agente con hasta diez intentos

No se puede afirmar que cada problema se resolverá a partir de una probabilidad. Bajo el muestreo sin reemplazo de las 20 soluciones, **P1 no se resuelve** (0), **P2** tiene probabilidad **0.763158**, **P3 0.983746**, **P4 0.999940**, y **P5 y P6** tienen probabilidad **1**: hay menos de diez fallos disponibles. Un verificador perfecto reconoce una solución correcta, pero no la genera. Estas garantías del conjunto finito no garantizan el éxito de diez nuevas generaciones independientes o de reintentos correlacionados con retroalimentación.

El promedio aumenta de **0.416667** en pass@1 a **0.791141** en pass@10 (**37.45 puntos porcentuales**). El modelo produce soluciones útiles que no siempre aparecen al primer intento; muestrear y verificar ayuda, pero no corrige P1, donde no se observó ningún éxito.